

PoultryTrak — Project Documentation

National Poultry Registration, Input Subsidy & Traceability System

Field	Value
Document version	1.0 (Draft for review)
Status	Design & Specification
Prepared for	Ministry of Food and Agriculture (MoFA) / Veterinary Services Directorate
Jurisdiction	Republic of Ghana
Stack	PHP 8.2+ · MySQL 8.0 · Carbon Design System (vanilla CSS/JS)
Last updated	2026-08-24

Table of contents

[Executive summary](#)

[Problem statement](#)

[Objectives and success metrics](#)

[Stakeholders and user roles](#)

[Scope](#)

[System architecture](#)

[Technology stack](#)

[Module 1 — Farmer registry & PoultryTrak ID](#)

[Module 2 — Leak-proof digital vouchers](#)

[Module 3 — QR-code traceability passport](#)

[Module 4 — Flock lifecycle & health records](#)

[Module 5 — Public verification portal](#)

[Module 6 — Administration, analytics & reporting](#)

[Data model](#)

[API specification](#)

[Front-end specification \(Carbon\)](#)

[Security architecture](#)

[Offline, USSD and low-connectivity strategy](#)

[Integrations](#)

[Non-functional requirements](#)

[Project structure & environment](#)

[Deployment & operations](#)

[Testing strategy](#)

[Governance, legal & data protection](#)

[Implementation roadmap](#)

[Risks and mitigations](#)

[Glossary](#)

1. Executive summary

PoultryTrak is an official digital system that does three things at once:

Registers genuine farmers. Every poultry farmer receives a unique, non-transferable **PoultryTrak ID**, tied to a national identity record and a GPS-verified farm location. If a farm does not exist on the ground, it does not exist in the register.

Secures subsidised inputs. Feed, day-old chicks, vaccines and veterinary medicine are released against **single-use, cryptographically signed digital vouchers** that can only be redeemed by the registered farmer, at an accredited dealer, within a defined window and quantity. This closes the diversion channels that drain conventional paper-coupon and list-based subsidy schemes.

Proves Ghanaian quality. Every bird leaving a registered farm carries a **QR-code passport** that resolves to a public record: the farmer's name and PoultryTrak ID, region and district of origin, hatch date, vaccination history, feed history, veterinary sign-off and processing date. Supermarkets, hotels, institutional buyers and export inspectors can verify a bird in under three seconds with any phone camera.

The result is a single spine of truth linking **who farmed it → what went into it → where it came from → who verified it**. That spine protects the farmer from input theft, protects the state from subsidy leakage, and protects the consumer and the export market from unverifiable product.

PoultryTrak is built as a mobile-first PHP/MySQL web application styled with IBM's Carbon Design System, chosen for its accessibility-first components, mature data-table and form patterns, and suitability for public-sector interfaces. It is designed to run on modest infrastructure, degrade gracefully to USSD and offline redemption on the 2G edges of the network, and remain operable by district officers with entry-level Android devices.

2. Problem statement

2.1 The current situation

Ghana's poultry sector imports the overwhelming majority of the chicken it consumes, while local producers operate under three structural handicaps that PoultryTrak addresses directly.

A. There is no reliable register of who actually farms. District-level records are held in spreadsheets, notebooks and association membership lists that do not reconcile with each other. Consequences: Subsidy programmes cannot target real producers, because the denominator is unknown.

"Briefcase farmers" — people who register a farm that does not exist, or register the same farm in several districts — capture allocations meant for producers.

Disease outbreak response is slow because the state cannot answer "which farms are within 5 km of this outbreak?" in less than several days.

Genuine farmers are invisible to lenders and insurers, because there is no verifiable production history to underwrite against.

B. Subsidised inputs leak before they reach the farm. Where inputs are allocated on a paper coupon or a printed beneficiary list, leakage occurs at predictable points:

Leak point	Mechanism
Allocation	Ghost beneficiaries added to the list; one farmer listed in multiple districts
Issuance	Coupons issued in bulk to intermediaries "on behalf of" farmers
Redemption	Coupon redeemed by someone other than the named farmer; no identity check at counter
Quantity	Dealer records full quantity, releases partial quantity, resells the balance
Reconciliation	Dealer claims reimbursement for coupons never matched to a real farmer
Resale	Subsidised feed re-enters the open market at commercial price

Every one of these is a *verification* failure, not a supply failure. The input exists; the proof of entitlement is forgeable.

C. Local chicken cannot prove what it is. A bird sold at a market or delivered to a supermarket carries no evidence of its origin, its vaccination status, its withdrawal-period compliance after antibiotic treatment, or its feed. This has direct commercial cost:

Formal buyers (supermarket chains, hotels, quick-service restaurants, institutional caterers) default to imported product because imports arrive with documentation.

Local producers cannot command a premium for quality they cannot evidence.

Export to regional and EU markets is blocked at the traceability requirement, not the quality requirement.

Food-safety incidents cannot be traced back to a source flock, so recalls are impossible and the whole sector absorbs the reputational damage of one bad actor.

2.2 What PoultryTrak changes

Before	After
Farmer identity asserted on paper	Farmer identity verified against national ID + GPS-verified farm polygon
Entitlement is a coupon anyone can carry	Entitlement is a signed token bound to one farmer, one dealer class, one quantity, one window
Redemption is a signature in a book	Redemption is an OTP/PIN-authenticated, geo-stamped, idempotent transaction
Reconciliation is a monthly reconciliation nightmare	Reconciliation is a real-time ledger
Bird origin is a verbal claim	Bird origin is a public, tamper-evident passport record
Outbreak tracing takes days	Outbreak tracing is a spatial query

3. Objectives and success metrics

3.1 Primary objectives

#	Objective
O1	Establish a single authoritative national register of poultry farmers and farm locations
O2	Deliver 100% of subsidised inputs to verified registered farmers, with a tamper-evident audit trail per unit
O3	Issue a verifiable origin-and-health passport for every bird leaving a registered farm
O4	Enable disease surveillance and outbreak tracing at district and national level
O5	Create bankable production histories that farmers can use to access credit and insurance
O6	Meet the traceability documentation threshold required by formal retail and export buyers

3.2 Success metrics

Metric	Baseline	Year 1 target	Year 3 target
Registered farms with GPS-verified location	0	15,000	60,000
Share of subsidised inputs redeemed via digital voucher	0%	70%	98%
Voucher leakage rate (redemptions failing post-audit reconciliation)	Unmeasured	< 3%	< 0.5%
Median voucher redemption time at dealer counter	n/a	< 90 s	< 45 s
Birds issued a QR passport	0	4 million	30 million
Passport scans by end buyers per month	0	25,000	400,000
Time to produce a list of farms within 5 km of an outbreak point	Days	< 60 s	< 10 s
Accredited dealers onboarded	0	400	1,500
Districts with active district officer accounts	0	120	261

3.3 Explicit non-goals

PoultryTrak is **not** an e-commerce marketplace, a farm-management ERP, a payment processor, or a veterinary diagnostic tool. It is a registry, an entitlement system, and a traceability ledger. It integrates with systems that do those other things; it does not replace them.

4. Stakeholders and user roles

4.1 Role matrix

Role	Who	Primary actions	Access scope
Farmer	Registered poultry producer	View PoultryTrak ID, view/redeem vouchers, record flock events, print passport labels	Own farms and flocks only
Farm worker	Delegate of a farmer	Record feed/vaccination events under supervision	One farm, event entry only, no voucher redemption
Extension officer	MoFA district agricultural extension agent	Register farmers, capture GPS, verify farms, approve applications, record inspections	Assigned district
Veterinary officer	Veterinary Services Directorate	Record vaccinations, treatments, withdrawal periods, issue health sign-off, flag quarantine	Assigned district/region
Accredited dealer	Agro-input shop, feed mill, hatchery	Validate and redeem vouchers, record stock movement, submit reimbursement claims	Own outlet(s)
Processor	Abattoir / processing plant	Receive flocks, link bird passports to processing batch, record slaughter date	Own facility
District administrator	District MoFA/veterinary lead	Manage allocations within district, approve dealers, view district dashboards	Assigned district
Regional administrator	Regional directorate	Regional allocation, oversight, escalation handling	Assigned region
National administrator	Programme secretariat	Programme setup, national allocation, dealer accreditation policy, user management	National
Auditor	Internal audit / Auditor-General / donor	Read-only access to all ledgers, exports, reconciliation reports	National, read-only

Role	Who	Primary actions	Access scope
Public verifier	Supermarket, hotel, exporter, consumer	Scan QR, view passport	Public passport data only, unauthenticated

4.2 Separation of duties

Three controls are enforced at the application layer and must not be configurable away:

No self-approval. The user who registers a farmer cannot be the user who verifies that farmer's farm location.

No issuance-and-redemption overlap. Dealer accounts cannot issue vouchers; administrator accounts cannot redeem them.

No retrospective silent edits. Every mutation of a registry, voucher or passport record writes an immutable audit row. Corrections are new rows, not overwrites.

5. Scope

5.1 In scope (v1.0)

Farmer registration, identity verification, farm GPS capture and polygon mapping

PoultryTrak ID issuance, lifecycle (active/suspended/revoked) and public verification

Programme, season and allocation configuration

Voucher issuance, distribution, offline-capable redemption, reversal and expiry

Dealer accreditation, outlet management, redemption terminal, reimbursement claims

Flock (batch) registration, feed events, vaccination events, treatment events with withdrawal tracking, mortality records

Bird passport generation, QR encoding, label/leg-band printing, batch and individual resolution

Public passport verification portal (no login)

District/regional/national dashboards, reconciliation reports, exports

Full audit log and role-based access control

USSD and SMS fallback for redemption and passport lookup

English-language UI with a translation layer prepared for Twi, Ewe, Ga, Dagbani and Hausa

5.2 Out of scope (v1.0, candidates for later phases)

Mobile money disbursement of cash grants

Marketplace / buyer-seller matching

Automated feed formulation advisory

IoT sensor ingestion (temperature, water, weight)

Blockchain anchoring of passport hashes (see §17.7 — architected for, not built in v1)

Full native mobile apps (v1 is a progressive web app)

Livestock other than poultry

5.3 Assumptions

Extension officers are issued Android devices with GPS and camera.

Accredited dealers have at least intermittent data connectivity and one shared Android device per outlet.

National identity verification is available via an integration or, failing that, via document capture and officer attestation.

The programme secretariat can enforce dealer accreditation and de-accreditation.

6. System architecture

6.1 Architectural overview



PUBLIC INTERNET

HTTPS (TLS 1.3)

Nginx — TLS, rate limit, WAF

FastCGI

PHP-FPM 8.2 (PoultryTrak)

Web (MVC) | REST API | USSD/SMS | Public Passport

Carbon UI | /api/v1 | gateway | /p/{code}

Domain layer

Registry · Voucher · Traceability · Health · Audit · Reporting

Data layer | (PDO, repositories)

MySQL 8 | Redis | Object | Queue | Cron worker
(primary) | sessions, | store | (DB-backed | expiry, recon,
+ replica | cache, | photos, | jobs) | reports, sync
| rate lim. | QR assets |

External integrations

National ID · SMS/USSD aggregator ·

Mobile money · GIS basemap · GS1

6.2 Layering rules

Layer	Responsibility	Must not
Presentation (<code>views/</code> , Carbon templates)	Render HTML, escape output, no business logic	Query the database directly
Controller (<code>app/Controllers/</code>)	Parse input, authorise, call services, choose a view or JSON response	Contain domain rules
Service / Domain (<code>app/Services/</code> , <code>app/Domain/</code>)	All business rules: eligibility, voucher state machine, passport assembly	Emit HTML or read <code>\$_POST</code>
Repository (<code>app/Repositories/</code>)	Prepared statements, mapping rows to entities, transactions	Contain business rules
Infrastructure (<code>app/Support/</code>)	Crypto, QR, SMS, storage, logging	Depend on controllers

6.3 Key architectural decisions

ID	Decision	Rationale
ADR-01	Server-rendered PHP with progressive enhancement, not an SPA	Works on low-end Android and 2G; no JS build pipeline required for core flows; smaller attack surface
ADR-02	Carbon Design System (vanilla, v10 CSS/JS) rather than Carbon React	Matches the server-rendered choice; no Node build step in the deployment path
ADR-03	Voucher entitlement carried as an HMAC-signed token, not just a database lookup	Dealers can verify authenticity offline; the database remains the settlement authority
ADR-04	Passport records are append-only with hash chaining per flock	Tamper evidence without the operational cost of a distributed ledger in v1
ADR-05	Bird passports are issued at flock granularity and <i>resolved</i> at bird granularity	Individually tagging live birds on-farm is not economically viable; individual identity is bound at processing/packing, and every individual code resolves to its flock's full history
ADR-06	Database-backed job queue rather than a broker	One fewer service to operate in a constrained environment; volumes do not justify Kafka/RabbitMQ
ADR-07	Soft multi-tenancy by district/region scope on every query	Single database, enforced tenancy via a mandatory scope filter in the repository base class

6.4 The ADR-05 clarification, stated plainly

The promise "every chicken gets a verifiable passport" is delivered as follows:

A **flock** (batch of birds housed together, sharing hatch date, feed, water and vaccination programme) is the unit of record. This is how the birds actually live and how disease and residue risk actually propagate.

Each flock has a **hash-chained event history**: chick source, every feed delivery, every vaccination, every treatment plus withdrawal period, mortality, inspections.

At exit — live-bird sale, slaughter or packing — the system mints **individual passport codes** from the flock, one per bird or per pack. Each code is unique, single-issue, and resolves to that flock's complete verified history plus the individual's exit date and destination.

Therefore: every chicken has its own scannable, unique, non-reusable code, and behind it sits a full, verifiable production record. Nothing is faked and nothing is over-claimed.

7. Technology stack

7.1 Core

Layer	Choice	Version	Notes
Language	PHP	8.2+	Typed properties, enums, readonly, <code>random_bytes</code> , <code>sodium_*</code>
Database	MySQL	8.0+	InnoDB, <code>utf8mb4</code> , CTEs, window functions, spatial types
Web server	Nginx + PHP-FPM	1.24 / 8.2	Apache + mod_php acceptable fallback
CSS / UI	Carbon Design System (vanilla)	v10.58	<code>carbon-components.min.css</code> , <code>carbon-components.min.js</code> , IBM Plex fonts
Cache / sessions	Redis	7.x	Optional: falls back to MySQL-backed sessions
Object storage	S3-compatible or local disk	—	Farm photos, QR label PDFs, ID document scans

7.2 PHP dependencies (Composer)

Package	Purpose
<code>endroid/qr-code</code>	QR generation (PNG/SVG), error correction level Q
<code>firebase/php-jwt</code>	Signed API tokens for the dealer terminal and mobile clients
<code>vlucas/phpdotenv</code>	Environment configuration
<code>monolog/monolog</code>	Structured application and audit logging
<code>phpmailer/phpmailer</code>	Transactional email (officers, dealers, admins)
<code>intervention/image</code>	Photo resize, EXIF strip, watermarking
<code>dompdf/dompdf</code>	Voucher slips, passport label sheets, reconciliation reports
<code>ramsey/uuid</code>	Idempotency keys, internal identifiers
<code>nesbot/carbon</code>	Date/time handling — note: unrelated to Carbon Design System, see below
<code>phpunit/phpunit</code> (dev)	Unit and integration tests
<code>squizlabs/php_codesniffer</code> (dev)	PSR-12 enforcement

Naming collision warning. "Carbon" in this document refers to two distinct things: **Carbon Design System** (IBM's CSS/UI framework, the front-end choice) and `nesbot/carbon` (a PHP date library). Where ambiguity is possible, the document says "Carbon DS" or "Carbon (dates)". Consider aliasing the date library on import to reduce confusion for new developers.

7.3 Why this stack

PHP + MySQL is the most widely available skill set in the Ghanaian developer market, meaning the system can be maintained and extended by local teams rather than a single vendor. It runs on inexpensive VPS or on-premise government infrastructure without container orchestration.

Carbon DS ships accessible components (WCAG 2.1 AA) out of the box — data tables, notifications, modals, form validation states, structured lists — which is exactly the vocabulary a registry and reconciliation UI needs. Its neutral, institutional aesthetic suits a government system, and its open licence carries no cost.

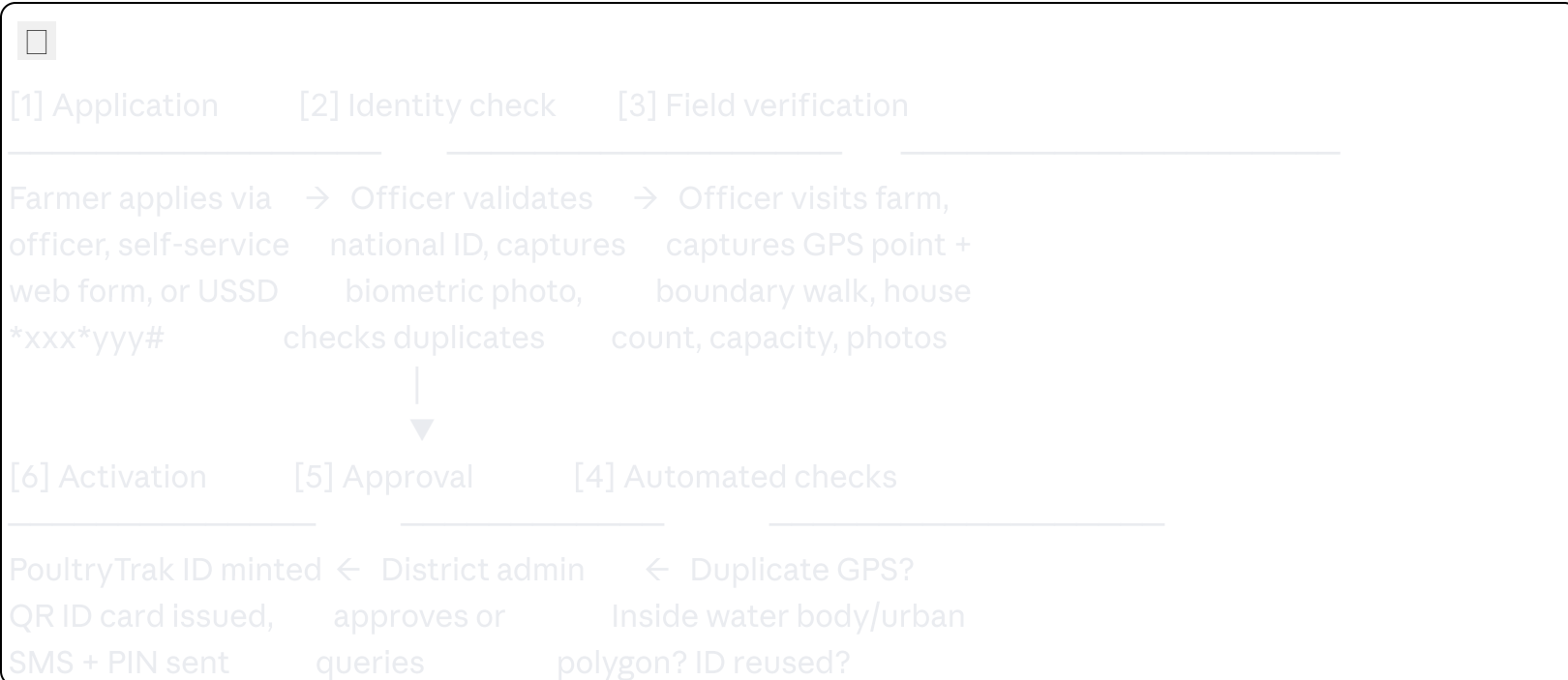
No SPA framework keeps the critical redemption path fast on low-end devices and removes a whole class of client-side state bugs from a system where correctness of a financial-adjacent transaction matters more than interaction flourish.

8. Module 1 — Farmer registry & PoultryTrak ID

8.1 Purpose

Establish, for every poultry producer in Ghana, a single verified identity that cannot be duplicated, cannot be transferred, and is anchored to a physical location that has been confirmed on the ground.

8.2 Registration flow



Step detail

Application. Captures personal details, contact number (MSISDN, verified by OTP), farm name, district, poultry type (broiler / layer / cockerel / guinea fowl / indigenous), current bird capacity, housing type, association membership.

Identity check. National ID number captured and, where the integration is live, validated against the national identity register; otherwise the ID document is photographed and attested by the officer. A face photo is

captured. The system runs a duplicate check on ID number, MSISDN and normalised name + date of birth.

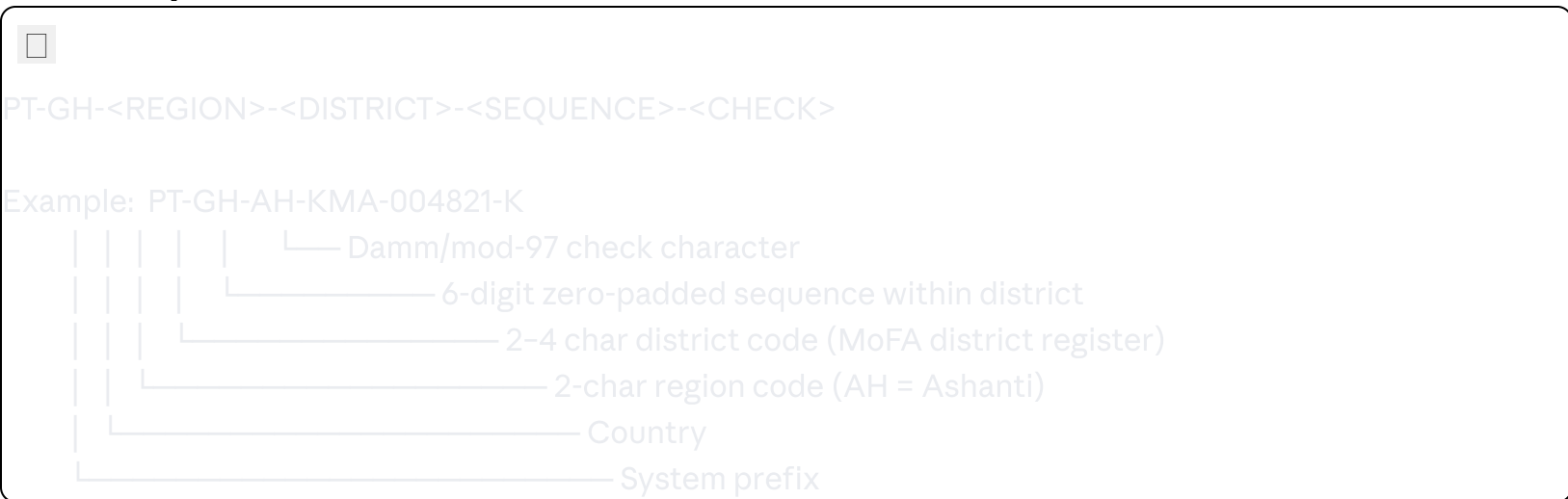
Field verification. The officer must be physically at the farm. The device captures a GPS point with accuracy metadata; readings above the configured accuracy threshold (default 25 m) are rejected. The officer optionally walks the perimeter to record a polygon. At least two geo-tagged photographs are required: one of the poultry house, one of the entrance/signage.

Automated checks. The system rejects or flags: a GPS point within 50 m of an already-verified farm belonging to a different farmer; a point falling inside a water body, forest reserve or dense-urban exclusion polygon; a national ID already linked to an active PoultryTrak ID; more than three farms per farmer without regional-level approval.

Approval. The district administrator reviews the dossier. Approval requires a different user account than the one that performed field verification (§4.2).

Activation. A PoultryTrak ID is minted, an ID card (PDF, printable, with QR) is generated, and the farmer receives an SMS containing the ID and a one-time PIN setup link/code.

8.3 PoultryTrak ID format



Properties:

Human-readable and dictatable over the phone or radio. Field officers and dealers must be able to read it aloud without error, so the check character catches single-character and transposition mistakes at the point of entry.

Never reassigned. Sequence numbers are consumed permanently; a revoked ID is never reissued to another person.

Not a secret. The ID identifies; it does not authenticate. Authentication is the farmer's PIN plus OTP. Any design that treats knowledge of the ID as proof of entitlement is invalid.

Region/district are the values at issuance. If a farmer relocates, the ID does not change; a `farm_transfers` record captures the movement, preserving history.

8.4 Farm records

A farmer may hold one or more farms. Each farm carries:

Field	Notes
farm_code	<PoultryTrakID>-F01, -F02, ...
Location point	POINT (SRID 4326), plus captured accuracy in metres
Boundary	Optional POLYGON, from perimeter walk
Verification status	pending → verified → re-verification_due / suspended
Verified by / at	Officer user ID and timestamp — never overwritten
Housing type	Deep litter, battery cage, free range, semi-intensive
Capacity	Maximum bird capacity, used to cap voucher entitlement
Biosecurity score	0-100, from the standard inspection checklist
Photos	Minimum 2, EXIF-stripped, stored with hash

Re-verification. Every farm requires re-verification every 24 months, or immediately on: capacity change above 25%, change of location, disease event, or a failed reconciliation flag. A farm past its re-verification date is automatically ineligible for new voucher allocation, though existing valid vouchers remain redeemable.

8.5 ID lifecycle states

State	Meaning	Can hold vouchers	Can redeem	Can issue passports
pending	Application submitted, not approved	No	No	No
active	Verified and in good standing	Yes	Yes	Yes
suspended	Under investigation, or re-verification overdue	Existing only	No	No
quarantined	Disease control order in force	Existing only	Yes (vaccines/meds only)	No
revoked	Fraud confirmed, or farmer deceased/exited	No	No	No
transferred	Farm handed to a successor ID	No	No	No

State transitions are permission-gated, require a reason code and free-text note, and are irreversible except by a role at least one level above the role that made the change.

9. Module 2 — Leak-proof digital vouchers

9.1 Design principle

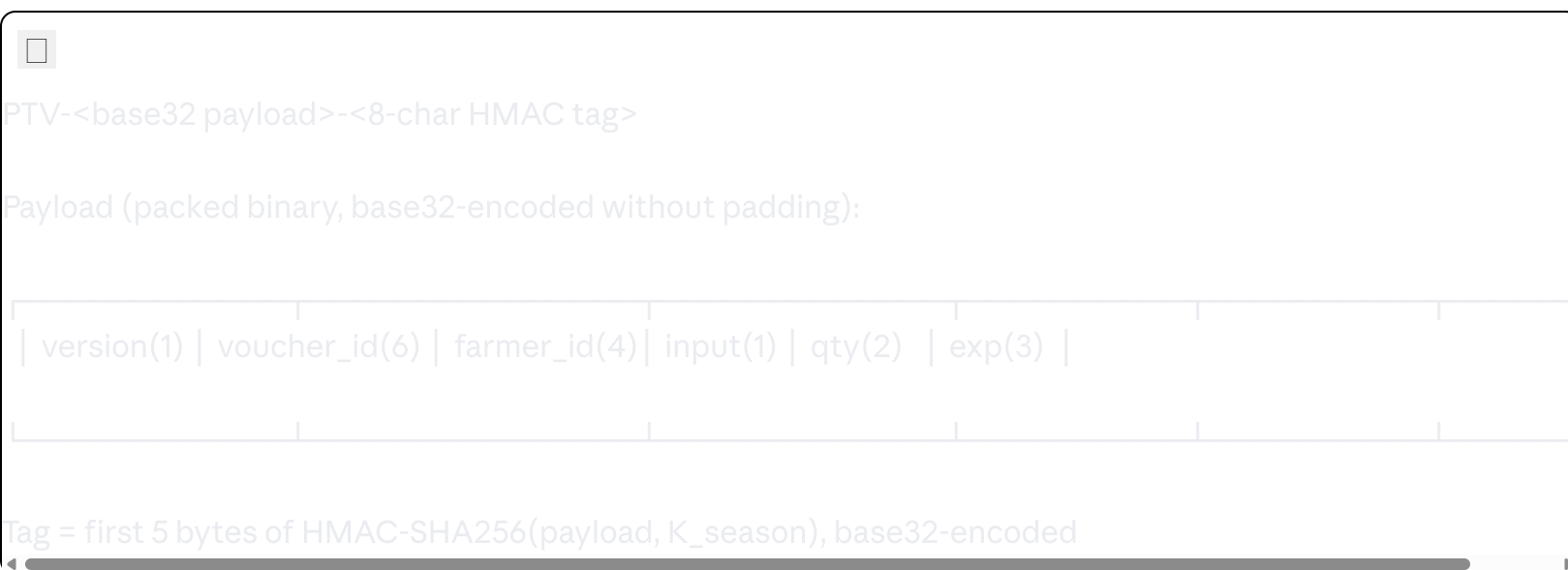
A voucher is not a document that grants entitlement. It is a *signed assertion about a specific farmer's entitlement*, which is worthless to anyone else and settles against a central ledger exactly once.

Five bindings make a voucher leak-resistant. All five must hold at redemption:

Binding	Enforced how
Bound to one farmer	Voucher carries <code>poultrytrak_id</code> ; redemption requires OTP to the registered MSISDN or the farmer's PIN entered on the dealer terminal
Bound to one input type and quantity	<code>input_type</code> + <code>quantity</code> + <code>unit</code> are inside the signed payload; partial redemption creates a residual voucher, it does not silently discard the balance
Bound to a dealer class and geography	Only accredited dealers of the matching class, within the farmer's region (or an explicitly permitted adjacent region), can redeem
Bound to a time window	<code>valid_from</code> / <code>valid_until</code> ; expiry returns the entitlement to the district pool automatically
Bound to single use	Atomic conditional update on the ledger row + idempotency key; a replayed request returns the original result rather than double-issuing

9.2 Voucher token structure

The voucher code presented to the dealer is compact enough to be read aloud or typed, and self-verifying offline:

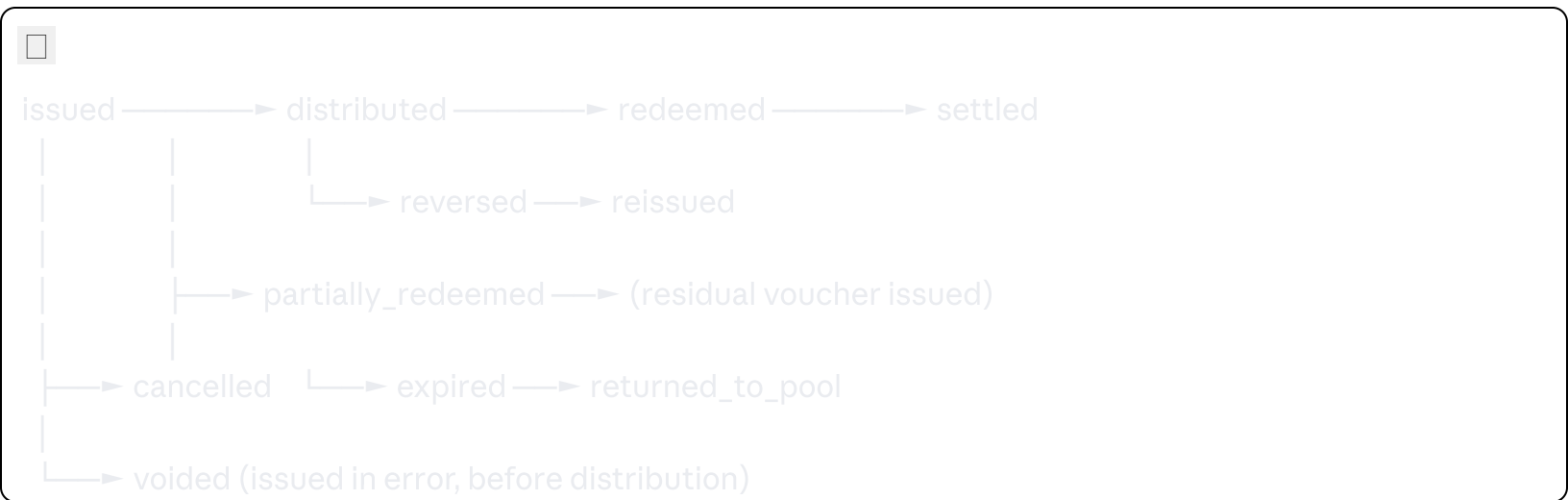


`K_season` is a per-programme-season signing key held server-side and distributed to dealer terminals in a device-bound, encrypted keystore for offline verification. It is rotated each season and can be revoked immediately.

Offline verification proves the voucher was *issued by the system*. It does **not** prove it is *unspent* — only the central ledger can do that. Offline redemption is therefore governed by strict limits (§18.2).

The QR encodes the same code plus a deep link, so dealers can scan rather than type.

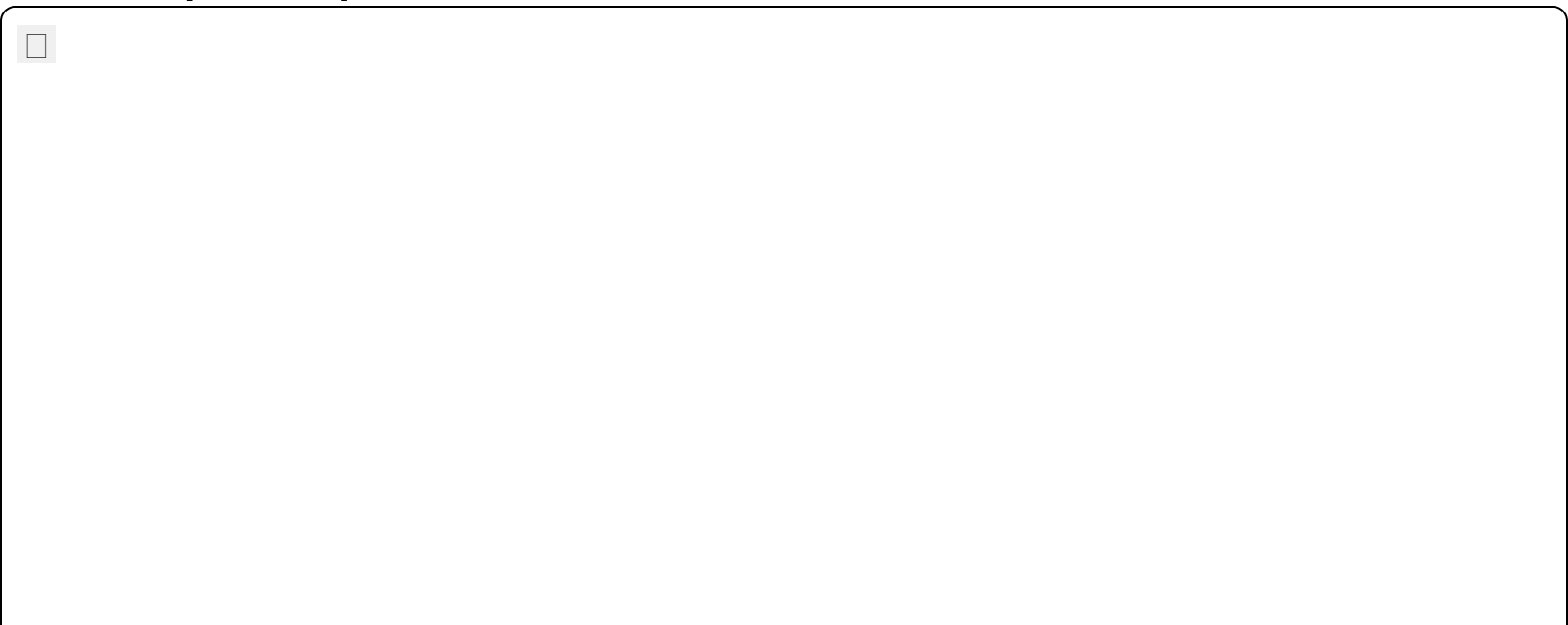
9.3 Voucher state machine



Transition	Trigger	Authority
<code>issued → distributed</code>	SMS/app delivery confirmed	System
<code>distributed → redeemed</code>	Successful dealer redemption	Dealer + farmer auth
<code>redeemed → reversed</code>	Dispute upheld, dealer error, fraud finding	District admin + reason code
<code>reversed → reissued</code>	New voucher minted for same entitlement	District admin
<code>distributed → expired</code>	<code>valid_until</code> passes	Cron job
<code>* → cancelled</code>	Programme change, farmer suspension	Regional/national admin

Reversal is deliberately **not** a delete. The redemption row stays; a compensating row is written. Reconciliation reports show both.

9.4 Redemption sequence (online)



Step 7 is the integrity heart of the system. It runs as a single transaction with row-level locking and an idempotency key stored under a unique index — a retried network call cannot produce two redemptions.

9.5 Anti-leakage controls

Control	Description
Capacity-linked entitlement	Maximum entitlement per season is a function of verified bird capacity and declared flock plan, not a flat per-farmer figure
Geofence check	Redemption GPS is compared to the dealer's registered outlet coordinates; deviation beyond a threshold flags the transaction for review
Device binding	Each dealer terminal is registered; a new device requires re-enrolment and admin approval
Velocity rules	More than N redemptions per outlet per hour, or an unusual share of redemptions to farmers outside the outlet's normal catchment, raises an alert
Farmer confirmation SMS	The farmer receives an SMS naming the dealer, input and quantity, with a short-code reply to dispute. Disputes freeze the dealer's claim line
Reconciliation before payment	Dealer reimbursement claims are paid only against redemptions that reconcile with stock movements and pass anomaly screening
Stock ledger	Dealers record deliveries in; redemptions deduct out. Redeeming more than stock on hand is impossible
Random verification	The system samples a percentage of redemptions each month for physical follow-up by extension officers; results feed a dealer risk score
No bulk redemption	A single account cannot redeem more than one farmer's voucher without a per-transaction farmer authentication event
Public spot-check	Farmers can view their own full redemption history in the app/USSD at any time, making silent redemption on their ID visible to them

9.6 Allocation and issuance

Allocation cascades: **National programme budget → regional allocation → district allocation → farmer entitlement**. Each level is capped by the level above; a district cannot issue vouchers worth more than its allocation. The system blocks the issuance rather than allowing an overrun to be discovered at reconciliation. Entitlement calculation (configurable per programme):

```
entitlement(input) = min(  
    capacity_factor(verified_capacity, input),  
    planned_flock_size × per_bird_norm(input),  
    programme_cap_per_farmer(input),  
    district_remaining_allocation(input)  
)
```

10. Module 3 — QR-code traceability passport

10.1 What a passport proves

A PoultryTrak passport is a public, read-only record that answers, for one bird or one pack:

Question	Source of truth
Who farmed it?	Verified farmer name + PoultryTrak ID
Where?	Region and district (settlement-level; precise coordinates are never public)
How old is it?	Hatch date from the flock record, sourced from a registered hatchery
Was it vaccinated?	Vaccination events signed off by a veterinary officer
Was it medicated, and is it safe?	Treatment events with withdrawal period, and a computed withdrawal-clear flag
What did it eat?	Feed events: type, brand, supplier, whether subsidised (voucher-linked)
Who inspected it?	Inspection and veterinary sign-off records
When did it leave the farm?	Exit event: date, destination class, processor
Is this code genuine?	Signature validation + single-issue check + scan history

10.2 Passport code and QR


Public URL: `https://verify.poultrytrak.gov.gh/p/{passport_code}`

passport_code = <flock_prefix><bird_serial><check>
e.g. 9F2KQ7 · 0004821 · M

QR payload: the full URL (so any generic camera app resolves it)

QR spec: Version auto, ECC level Q (25% recovery), quiet zone 4 modules,
minimum printed size 18 mm × 18 mm, black on white or high-contrast
equivalent, tested against smudging on leg bands and pack film

Design constraints that matter in practice:

Codes are non-sequential and non-guessable in their public form. The visible code is derived through a keyed permutation of the internal ID, so a scraper cannot walk the range and harvest the register.

ECC level Q is chosen because labels get wet, greasy, folded and abraded in a poultry environment. Level M is not enough; level H costs too much module density at 18 mm.

Single-issue. A passport code is minted once and never reissued. Two identical codes appearing in the field is, by construction, evidence of counterfeiting, and the scan-anomaly detector treats it as such.

Offline-degraded reading. The code is also printed in human-readable form beneath the QR, and can be checked by SMS short code, so a buyer with no data can still verify.

10.3 Passport issuance points

Point	Carrier	Granularity
Live bird sale at farm gate	Leg band or wing tag	Per bird
Delivery to processor	Consignment note QR + per-crate labels	Per crate, expandable to per bird
Post-processing pack	Label on primary pack	Per pack (whole bird or portions)
Table egg tray (<i>phase 2</i>)	Tray label	Per tray, per lay-date

At each point the system records an **exit event** binding the individual code to its flock, so the chain remains unbroken through processing.

10.4 What is public and what is not

Public (visible on scan)	Restricted (never public)
Farmer display name, PoultryTrak ID	Farmer phone number, national ID, address
Region, district, nearest town	Exact farm GPS coordinates
Hatch date, exit date, age at exit	Farm capacity, financials, voucher values
Vaccination types and dates	Individual health complaints, non-notifiable incidents
Feed type and supplier class	Contract prices
Withdrawal-period compliance flag	Raw treatment dosage records
Veterinary sign-off status and officer ID	Officer personal contact details
Processor name and processing date	Buyer identity

Precise farm coordinates are withheld deliberately: publishing them would expose farmers to theft and would create a targeting map for anyone wishing to harm the sector.

10.5 Tamper evidence

Each flock maintains a hash chain over its event log:



```
event_hash(n) = SHA-256( event_hash(n-1) || canonical_json(event(n)) )
```

Every event stores its own hash and the previous hash.

The flock header stores the current head hash.

A nightly job recomputes each active flock's chain and raises a critical alert on mismatch.

Head hashes for closed flocks are batched daily into a Merkle root that is published (and, in a later phase, anchored externally — see §17.7). Once a root is published, retroactively altering any event in that batch is detectable by anyone who kept the root.

This gives verifiable integrity without the operational burden of a distributed ledger, and leaves the door open to anchoring later without redesigning the data model.

10.6 The public scan experience

A buyer scanning a code sees, in this order, on one screen:

A verdict banner — green VERIFIED, amber VERIFIED WITH NOTICE, red NOT VERIFIED. Carbon notification components carry this.

Origin card — farmer name, PoultryTrak ID, district, region, with a small map showing the district (not the farm).

Health card — vaccination timeline, withdrawal-period status, veterinary sign-off.

Feed card — feed types and suppliers across the rearing period.

Journey timeline — hatch → rearing → exit → processing, with dates.

Verification metadata — first scan, total scans, and a warning if the scan pattern is anomalous.

Amber and red states must be as clear as green. A system that only shows good news is not a verification system.

11. Module 4 — Flock lifecycle & health records

11.1 Flock registration

A flock is opened when birds arrive on the farm:

Field	Required	Notes
Farm	Yes	Must be verified
Flock code	Auto	<farm_code>-B<YYMM><seq>
Bird type / breed	Yes	Broiler, layer, cockerel, guinea fowl, indigenous
Source hatchery	Yes	Registered hatchery, or imported with permit reference
Chick voucher reference	If applicable	Links subsidised chicks to this flock
Quantity placed	Yes	Number of day-old chicks or point-of-lay birds
Hatch date / placement date	Yes	
House / pen	Yes	
Planned exit date	Yes	Drives feed norms and voucher entitlement

11.2 Events recorded during the cycle

Event	Captured by	Key fields
Feed delivery	Farmer / worker	Feed type (starter/grower/finisher/layer mash), brand, supplier, quantity, subsidy voucher ref, batch number
Vaccination	Vet officer (or farmer with vet confirmation)	Vaccine, disease (Newcastle, Gumboro, Fowl Pox, Infectious Bronchitis, Marek's), batch no., route, date, administering officer
Treatment	Vet officer	Product, active ingredient, indication, dose, start/end date, withdrawal period in days , computed withdrawal-clear date
Mortality	Farmer	Count, suspected cause, disposal method
Weight sampling	Farmer	Sample size, average weight, uniformity
Inspection	Extension / vet officer	Biosecurity checklist score, findings, photos
Disease report	Farmer or officer	Signs, count affected, escalation to notifiable-disease workflow
Exit	Farmer + processor/buyer	Count, destination class, buyer/processor reference

11.3 Withdrawal-period enforcement

This is the food-safety control that gives the passport its commercial value:



```
IF treatment.withdrawal_clear_date > proposed_exit_date THEN
  BLOCK passport issuance for this flock
  RAISE notice to farmer and district vet officer
  ALLOW override ONLY by a veterinary officer, with a written justification,
  which is recorded and shown on the public passport as a notice
```

A flock exiting inside a withdrawal period cannot silently receive a green verdict. Either the exit is delayed, or the passport carries an amber notice that a buyer can see. This is non-negotiable in the design: it is the difference between a traceability system and a marketing badge.

11.4 Disease surveillance

Any report of a **notifiable disease** (e.g. highly pathogenic avian influenza, Newcastle disease outbreak) immediately: flags the flock, notifies the district and regional veterinary officers, and runs a spatial query for all farms within a configurable radius.

The spatial query uses MySQL's `ST_Distance_Sphere` against the farms table with a spatial index, returning contact-tracing candidates in seconds.

A quarantine order sets affected farms to `quarantined` (§8.5): passport issuance stops; vaccine and medicine vouchers continue to work.

12. Module 5 — Public verification portal

12.1 Endpoints

Path	Purpose	Auth
<code>/p/{passport_code}</code>	Bird/pack passport view	None
<code>/f/{poultrytrak_id}</code>	Farmer verification — confirms an ID is real, active and in which district	None
<code>/d/{dealer_code}</code>	Dealer accreditation check — is this shop authorised?	None
<code>/verify</code>	Manual code entry for people who cannot scan	None
<code>/bulk</code>	CSV upload of codes for supermarkets/exporters verifying a consignment API key	

12.2 Abuse controls on public endpoints

Public verification is a scraping target, so:

Rate limiting per IP and per code (Redis token bucket), with a friendly Carbon inline notification rather than a bare 429 page.

Codes are non-enumerable (§10.2); a sequential-scan pattern triggers throttling and an alert.

No personally identifying data beyond the display name and ID (§10.4).

Bulk verification requires an API key issued to a named organisation, with quotas and logging.

Scan telemetry (timestamp, coarse geography, user agent) is recorded for anomaly detection — for example, the same code scanned in Tamale and Takoradi within an hour, which suggests a duplicated label.

12.3 Buyer-facing features

Consignment verification. A supermarket receiving 500 packs uploads the code list; the portal returns a pass/fail summary with reasons, exportable as a PDF for the buyer's own quality file.

Exporter dossier. For an export consignment, the system compiles a signed PDF pack: flock histories, vaccination records, veterinary sign-off, withdrawal compliance, processing dates — the documentation set buyers and inspectors ask for.

Embeddable badge. A retailer can display a "PoultryTrak Verified" badge that links to live verification, so the claim is checkable rather than decorative.

13. Module 6 — Administration, analytics & reporting

13.1 Dashboards

Dashboard	Audience	Key widgets
National	Programme secretariat	Registrations by region, allocation vs issuance vs redemption, leakage indicators, passport issuance trend, active outbreaks
Regional	Regional director	District league table, dealer performance, redemption velocity, re-verification backlog
District	District admin	Farms pending verification, vouchers expiring in 7 days, disputes open, inspections due, quarantine status
Dealer	Accredited dealer	Redemptions today, stock on hand, claims pending/paid, risk score
Farmer	Farmer	My vouchers, my flocks, my passports, my redemption history

13.2 Standard reports

Report	Frequency	Contents
Voucher reconciliation	Daily / monthly	Issued, redeemed, expired, reversed, by district, dealer and input type; unmatched items listed individually
Dealer claim schedule	Per payment cycle	Redemptions eligible for reimbursement, exclusions with reasons
Leakage & anomaly report	Weekly	Geofence violations, velocity flags, disputed redemptions, duplicate-scan alerts, dealer risk ranking
Registration integrity	Monthly	Duplicate candidates, farms overdue re-verification, GPS accuracy distribution
Traceability coverage	Monthly	Share of exits with passports issued, scan rates by buyer class
Animal health	Monthly	Vaccination coverage by disease and district, mortality rates, withdrawal-period overrides
Audit extract	On demand	Full immutable log for a date range, CSV + checksum

13.3 Data export

All reports export to CSV and PDF. Every export writes an audit row naming the user, the filter set and the row count, and every exported file carries a footer with a generation timestamp and a checksum, so a document circulating outside the system can be checked against it.

14. Data model

14.1 Entity relationship overview





14.2 Core schema (MySQL 8.0)

sql

-- REFERENCE

```
CREATE TABLE regions (  
  id      TINYINT UNSIGNED PRIMARY KEY,  
  code    CHAR(2)  NOT NULL UNIQUE,    -- 'AH', 'GA', ...  
  name    VARCHAR(80) NOT NULL,  
  created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

```
CREATE TABLE districts (  
  id      SMALLINT UNSIGNED PRIMARY KEY,  
  region_id  TINYINT UNSIGNED NOT NULL,  
  code    VARCHAR(4) NOT NULL,  
  name    VARCHAR(120) NOT NULL,  
  capital  VARCHAR(120) NULL,  
  UNIQUE KEY uq_district_code (region_id, code),  
  CONSTRAINT fk_district_region FOREIGN KEY (region_id) REFERENCES regions(id)  
) ENGINE=InnoDB;
```

-- IDENTITY & ACCESS

```
CREATE TABLE users (  
  id      BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,  
  uuid    CHAR(36)  NOT NULL UNIQUE,  
  full_name  VARCHAR(160) NOT NULL,  
  email    VARCHAR(190) NULL UNIQUE,  
  msisdn   VARCHAR(20) NOT NULL UNIQUE,    -- E.164, e.g. +233XXXXXXXXXX  
  password_hash  VARCHAR(255) NULL,        -- Argon2id  
  pin_hash  VARCHAR(255) NULL,            -- farmers/dealers, Argon2id  
  status    ENUM('active','suspended','locked','disabled') NOT NULL DEFAULT 'active',  
  region_id  TINYINT UNSIGNED NULL,        -- scope for officers/admins  
  district_id  SMALLINT UNSIGNED NULL,  
  mfa_secret  VARBINARY(255) NULL,        -- TOTP, encrypted at rest  
  failed_attempts  TINYINT UNSIGNED NOT NULL DEFAULT 0,  
  locked_until  DATETIME NULL,  
  last_login_at  DATETIME NULL,  
  created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  updated_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  INDEX idx_users_scope (region_id, district_id),
```

```

INDEX idx_users_status (status)
) ENGINE=InnoDB;

CREATE TABLE roles (
  id      TINYINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  slug    VARCHAR(40) NOT NULL UNIQUE, -- farmer, extension_officer, vet_officer, ...
  name    VARCHAR(80) NOT NULL,
  scope_level ENUM('self','outlet','district','region','national') NOT NULL
) ENGINE=InnoDB;

CREATE TABLE user_roles (
  user_id  BIGINT UNSIGNED NOT NULL,
  role_id  TINYINT UNSIGNED NOT NULL,
  granted_by  BIGINT UNSIGNED NULL,
  granted_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (user_id, role_id),
  CONSTRAINT fk_ur_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  CONSTRAINT fk_ur_role FOREIGN KEY (role_id) REFERENCES roles(id)
) ENGINE=InnoDB;

CREATE TABLE permissions (
  id      SMALLINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  slug    VARCHAR(60) NOT NULL UNIQUE -- voucher.issue, farm.verify, passport.mint
) ENGINE=InnoDB;

CREATE TABLE role_permissions (
  role_id  TINYINT UNSIGNED NOT NULL,
  permission_id SMALLINT UNSIGNED NOT NULL,
  PRIMARY KEY (role_id, permission_id)
) ENGINE=InnoDB;

-- =====
-- FARMER REGISTRY
-- =====

CREATE TABLE farmers (
  id      BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  poultrytrak_id  VARCHAR(28) NOT NULL UNIQUE, -- PT-GH-AH-KMA-004821-K
  user_id  BIGINT UNSIGNED NOT NULL UNIQUE,
  full_name  VARCHAR(160) NOT NULL,
  display_name  VARCHAR(160) NOT NULL, -- shown publicly on passports
  date_of_birth  DATE NULL,

```

```

gender          ENUM('female','male','other','undisclosed') NULL,
national_id_hash CHAR(64) NOT NULL,          -- SHA-256(salted) for dedupe
national_id_enc  VARBINARY(512) NOT NULL,     -- AES-256-GCM, KMS-managed key
national_id_verified TINYINT(1) NOT NULL DEFAULT 0,
region_id       TINYINT UNSIGNED NOT NULL,
district_id     SMALLINT UNSIGNED NOT NULL,
association     VARCHAR(120) NULL,
status         ENUM('pending','active','suspended','quarantined','revoked','transferred')
              NOT NULL DEFAULT 'pending',
status_reason   VARCHAR(255) NULL,
registered_by   BIGINT UNSIGNED NULL,
approved_by    BIGINT UNSIGNED NULL,
approved_at    DATETIME NULL,
created_at     TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
updated_at     TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
UNIQUE KEY uq_national_id (national_id_hash),
INDEX idx_farmers_scope (region_id, district_id, status),
CONSTRAINT fk_farmer_user FOREIGN KEY (user_id) REFERENCES users(id),
CONSTRAINT fk_farmer_district FOREIGN KEY (district_id) REFERENCES districts(id),
CONSTRAINT chk_approver_differs CHECK (approved_by IS NULL OR approved_by <> registered_by)
) ENGINE=InnoDB;

```

```

CREATE TABLE farms (
  id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  farmer_id   BIGINT UNSIGNED NOT NULL,
  farm_code   VARCHAR(34) NOT NULL UNIQUE,    -- PT-GH-AH-KMA-004821-K-F01
  name        VARCHAR(160) NOT NULL,
  district_id SMALLINT UNSIGNED NOT NULL,
  locality    VARCHAR(160) NULL,
  location    POINT SRID 4326 NOT NULL,       -- captured GPS point
  gps_accuracy_m DECIMAL(6,2) NOT NULL,
  boundary    POLYGON SRID 4326 NULL,        -- optional perimeter walk
  housing_type ENUM('deep_litter','battery_cage','free_range','semi_intensive') NOT NULL,
  bird_capacity INT UNSIGNED NOT NULL,
  biosecurity_score TINYINT UNSIGNED NULL,
  verification_status ENUM('pending','verified','reverification_due','suspended')
                  NOT NULL DEFAULT 'pending',
  verified_by   BIGINT UNSIGNED NULL,
  verified_at   DATETIME NULL,
  reverify_due_at DATE NULL,
  created_at    TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

```

```
updated_at    TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
SPATIAL INDEX idx_farm_location (location),
INDEX idx_farm_farmer (farmer_id),
INDEX idx_farm_verification (verification_status, reverify_due_at),
CONSTRAINT fk_farm_farmer FOREIGN KEY (farmer_id) REFERENCES farmers(id)
) ENGINE=InnoDB;
```

```
CREATE TABLE farm_verifications (
  id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  farm_id     BIGINT UNSIGNED NOT NULL,
  officer_id  BIGINT UNSIGNED NOT NULL,
  outcome     ENUM('verified','rejected','deferred') NOT NULL,
  location    POINT SRID 4326 NOT NULL, -- where the OFFICER stood
  accuracy_m  DECIMAL(6,2) NOT NULL,
  notes       TEXT NULL,
  photo_refs  JSON NOT NULL,          -- [{path, sha256, taken_at, lat, lng}]
  device_id   VARCHAR(64) NULL,
  created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_fv_farm (farm_id, created_at),
  CONSTRAINT fk_fv_farm FOREIGN KEY (farm_id) REFERENCES farms(id)
) ENGINE=InnoDB;
```

14.3 Voucher schema



sql

```

CREATE TABLE programmes (
  id          SMALLINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  code        VARCHAR(24) NOT NULL UNIQUE,      -- 'BROILER-2026A'
  name        VARCHAR(160) NOT NULL,
  season_year  SMALLINT UNSIGNED NOT NULL,
  starts_on   DATE NOT NULL,
  ends_on     DATE NOT NULL,
  signing_key_id VARCHAR(32) NOT NULL,          -- reference into KMS, never the key
  status      ENUM('draft','active','closed') NOT NULL DEFAULT 'draft',
  rules       JSON NOT NULL                    -- entitlement norms, caps, input classes
) ENGINE=InnoDB;

```

```

CREATE TABLE input_types (
  id          SMALLINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  code        VARCHAR(24) NOT NULL UNIQUE,      -- FEED_STARTER, DOC_BROILER, VAC_NDV
  category    ENUM('feed','chick','vaccine','medicine','equipment') NOT NULL,
  name        VARCHAR(120) NOT NULL,
  unit        VARCHAR(16) NOT NULL,             -- kg, bag, dose, bird
  unit_value_ghs DECIMAL(12,2) NOT NULL,
  subsidy_pct DECIMAL(5,2) NOT NULL
) ENGINE=InnoDB;

```

```

CREATE TABLE allocations (
  id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  programme_id SMALLINT UNSIGNED NOT NULL,
  level       ENUM('national','region','district') NOT NULL,
  region_id   TINYINT UNSIGNED NULL,
  district_id SMALLINT UNSIGNED NULL,
  input_type_id SMALLINT UNSIGNED NOT NULL,
  quantity    DECIMAL(14,3) NOT NULL,
  quantity_used DECIMAL(14,3) NOT NULL DEFAULT 0,
  created_by   BIGINT UNSIGNED NOT NULL,
  created_at   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uq_alloc (programme_id, level, region_id, district_id, input_type_id),
  CONSTRAINT chk_alloc_not_overdrawn CHECK (quantity_used <= quantity)
) ENGINE=InnoDB;

```

```

CREATE TABLE dealers (
  id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  dealer_code  VARCHAR(20) NOT NULL UNIQUE,     -- PTD-AH-0042
  business_name VARCHAR(180) NOT NULL,

```

```

tin          VARCHAR(30) NULL,
classes      SET('feed','chick','vaccine','medicine','equipment') NOT NULL,
status       ENUM('pending','accredited','suspended','revoked') NOT NULL DEFAULT 'pending',
risk_score   TINYINT UNSIGNED NOT NULL DEFAULT 0, -- 0 low .. 100 high
accredited_at DATETIME NULL,
created_at   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

```

```

CREATE TABLE dealer_outlets (
id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
dealer_id   BIGINT UNSIGNED NOT NULL,
outlet_code VARCHAR(24) NOT NULL UNIQUE,
district_id SMALLINT UNSIGNED NOT NULL,
location    POINT SRID 4326 NOT NULL,
geofence_m  SMALLINT UNSIGNED NOT NULL DEFAULT 300,
status      ENUM('active','suspended') NOT NULL DEFAULT 'active',
SPATIAL INDEX idx_outlet_location (location),
CONSTRAINT fk_outlet_dealer FOREIGN KEY (dealer_id) REFERENCES dealers(id)
) ENGINE=InnoDB;

```

```

CREATE TABLE dealer_devices (
id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
outlet_id   BIGINT UNSIGNED NOT NULL,
device_id   VARCHAR(64) NOT NULL UNIQUE,
public_key  VARBINARY(512) NULL,
enrolled_by BIGINT UNSIGNED NOT NULL,
enrolled_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
status      ENUM('active','revoked') NOT NULL DEFAULT 'active',
last_seen_at DATETIME NULL
) ENGINE=InnoDB;

```

```

CREATE TABLE vouchers (
id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
voucher_code VARCHAR(48) NOT NULL UNIQUE, -- PTV-XXXXXXXX-XXXXXXXX
programme_id SMALLINT UNSIGNED NOT NULL,
farmer_id   BIGINT UNSIGNED NOT NULL,
farm_id     BIGINT UNSIGNED NOT NULL,
input_type_id SMALLINT UNSIGNED NOT NULL,
quantity    DECIMAL(12,3) NOT NULL,
quantity_redeemed DECIMAL(12,3) NOT NULL DEFAULT 0,
value_ghs   DECIMAL(12,2) NOT NULL,

```

```

allowed_region_id TINYINT UNSIGNED NOT NULL,
status          ENUM('issued','distributed','partially_redeemed','redeemed',
                    'expired','reversed','cancelled','voided') NOT NULL DEFAULT 'issued',
parent_voucher_id BIGINT UNSIGNED NULL,          -- residual after partial redemption
valid_from      DATE NOT NULL,
valid_until     DATE NOT NULL,
issued_by       BIGINT UNSIGNED NOT NULL,
issued_at       TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
distributed_at   DATETIME NULL,
closed_at       DATETIME NULL,
INDEX idx_voucher_farmer (farmer_id, status),
INDEX idx_voucher_expiry (status, valid_until),
INDEX idx_voucher_programme (programme_id, input_type_id),
CONSTRAINT fk_voucher_farmer FOREIGN KEY (farmer_id) REFERENCES farmers(id),
CONSTRAINT chk_voucher_qty CHECK (quantity_redeemed <= quantity)
) ENGINE=InnoDB;

```

```

CREATE TABLE voucher_redemptions (
  id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  voucher_id   BIGINT UNSIGNED NOT NULL,
  outlet_id    BIGINT UNSIGNED NOT NULL,
  device_id    VARCHAR(64) NULL,
  redeemed_by  BIGINT UNSIGNED NOT NULL,          -- dealer user
  quantity     DECIMAL(12,3) NOT NULL,
  value_ghs    DECIMAL(12,2) NOT NULL,
  auth_method  ENUM('otp','pin','offline_signed') NOT NULL,
  location     POINT SRID 4326 NULL,
  geofence_ok  TINYINT(1) NOT NULL DEFAULT 1,
  idempotency_key CHAR(36) NOT NULL,
  mode         ENUM('online','offline_synced') NOT NULL DEFAULT 'online',
  occurred_at  DATETIME NOT NULL,                  -- real-world time (may precede sync)
  recorded_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  reversed_at  DATETIME NULL,
  reversal_reason VARCHAR(255) NULL,
  UNIQUE KEY uq_idempotency (idempotency_key),
  INDEX idx_redemption_voucher (voucher_id),
  INDEX idx_redemption_outlet (outlet_id, occurred_at),
  CONSTRAINT fk_red_voucher FOREIGN KEY (voucher_id) REFERENCES vouchers(id)
) ENGINE=InnoDB;

```

```

CREATE TABLE dealer_claims (

```

```

id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
dealer_id   BIGINT UNSIGNED NOT NULL,
period_start DATE NOT NULL,
period_end  DATE NOT NULL,
gross_ghs   DECIMAL(14,2) NOT NULL,
excluded_ghs DECIMAL(14,2) NOT NULL DEFAULT 0,
payable_ghs DECIMAL(14,2) NOT NULL,
status      ENUM('draft','submitted','under_review','approved','paid','rejected')
            NOT NULL DEFAULT 'draft',
reviewed_by BIGINT UNSIGNED NULL,
paid_reference VARCHAR(64) NULL,
created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
UNIQUE KEY uq_claim_period (dealer_id, period_start, period_end)
) ENGINE=InnoDB;

```

```

CREATE TABLE disputes (
id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
redemption_id BIGINT UNSIGNED NULL,
farmer_id   BIGINT UNSIGNED NOT NULL,
channel     ENUM('sms','ussd','web','officer') NOT NULL,
category    ENUM('not_received','wrong_quantity','not_me','quality','other') NOT NULL,
description TEXT NULL,
status      ENUM('open','investigating','upheld','rejected','withdrawn')
            NOT NULL DEFAULT 'open',
resolved_by BIGINT UNSIGNED NULL,
resolution_note TEXT NULL,
created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
resolved_at DATETIME NULL,
INDEX idx_dispute_status (status, created_at)
) ENGINE=InnoDB;

```

14.4 Traceability schema



sql

```

CREATE TABLE flocks (
  id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  flock_code   VARCHAR(40) NOT NULL UNIQUE,
  farm_id      BIGINT UNSIGNED NOT NULL,
  bird_type    ENUM('broiler','layer','cockerel','guinea_fowl','indigenous') NOT NULL,
  breed        VARCHAR(80) NULL,
  hatchery_id  BIGINT UNSIGNED NULL,
  chick_voucher_id BIGINT UNSIGNED NULL,
  quantity_placed INT UNSIGNED NOT NULL,
  quantity_current INT UNSIGNED NOT NULL,
  placement_date DATE NOT NULL,
  planned_exit_date DATE NOT NULL,
  status       ENUM('active','closed','quarantined','condemned') NOT NULL DEFAULT 'active',
  chain_head_hash CHAR(64) NULL,          -- head of the event hash chain
  withdrawal_clear_at DATE NULL,          -- latest clear date across treatments
  created_at    TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_flock_farm (farm_id, status),
  CONSTRAINT fk_flock_farm FOREIGN KEY (farm_id) REFERENCES farms(id)
) ENGINE=InnoDB;

```

```

CREATE TABLE flock_events (
  id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  flock_id    BIGINT UNSIGNED NOT NULL,
  sequence_no INT UNSIGNED NOT NULL,
  event_type  ENUM('placement','feed','vaccination','treatment','mortality',
                  'weight','inspection','disease_report','exit','note') NOT NULL,
  occurred_on DATE NOT NULL,
  payload     JSON NOT NULL,          -- type-specific, schema-validated in app layer
  recorded_by BIGINT UNSIGNED NOT NULL,
  vet_signed_by BIGINT UNSIGNED NULL, -- required for vaccination/treatment
  prev_hash   CHAR(64) NULL,
  event_hash  CHAR(64) NOT NULL,
  created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uq_flock_seq (flock_id, sequence_no),
  INDEX idx_event_type (flock_id, event_type, occurred_on),
  CONSTRAINT fk_event_flock FOREIGN KEY (flock_id) REFERENCES flocks(id)
) ENGINE=InnoDB;

```

```

CREATE TABLE flock_exits (
  id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  flock_id    BIGINT UNSIGNED NOT NULL,

```

```

exit_type    ENUM('live_sale','processor','own_use','mortality_disposal') NOT NULL,
quantity     INT UNSIGNED NOT NULL,
processor_id BIGINT UNSIGNED NULL,
buyer_ref    VARCHAR(120) NULL,
exit_date    DATE NOT NULL,
withdrawal_ok TINYINT(1) NOT NULL,
override_by  BIGINT UNSIGNED NULL,    -- vet override if withdrawal not clear
override_note TEXT NULL,
created_at   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
INDEX idx_exit_flock (flock_id, exit_date)
) ENGINE=InnoDB;

```

```

CREATE TABLE passports (
  id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  passport_code VARCHAR(24) NOT NULL UNIQUE,    -- public, non-enumerable
  flock_id    BIGINT UNSIGNED NOT NULL,
  exit_id     BIGINT UNSIGNED NULL,
  unit_type   ENUM('bird','crate','pack','tray') NOT NULL DEFAULT 'bird',
  unit_index  INT UNSIGNED NULL,
  verdict     ENUM('verified','notice','not_verified') NOT NULL DEFAULT 'verified',
  notice_reason VARCHAR(255) NULL,
  snapshot    JSON NOT NULL,    -- frozen public record at minting time
  snapshot_hash CHAR(64) NOT NULL,
  minted_by   BIGINT UNSIGNED NOT NULL,
  minted_at   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  first_scan_at DATETIME NULL,
  scan_count  INT UNSIGNED NOT NULL DEFAULT 0,
  INDEX idx_passport_flock (flock_id),
  INDEX idx_passport_verdict (verdict),
  CONSTRAINT fk_passport_flock FOREIGN KEY (flock_id) REFERENCES flocks(id)
) ENGINE=InnoDB;

```

```

CREATE TABLE passport_scans (
  id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  passport_id BIGINT UNSIGNED NOT NULL,
  scanned_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  ip_hash     CHAR(64) NULL,
  coarse_region VARCHAR(80) NULL,
  user_agent  VARCHAR(255) NULL,
  channel     ENUM('web','sms','api','bulk') NOT NULL DEFAULT 'web',
  anomaly_flag TINYINT(1) NOT NULL DEFAULT 0,

```

```

INDEX idx_scan_passport (passport_id, scanned_at)
) ENGINE=InnoDB
PARTITION BY RANGE (YEAR(scanned_at)) (
  PARTITION p2026 VALUES LESS THAN (2027),
  PARTITION p2027 VALUES LESS THAN (2028),
  PARTITION pmax VALUES LESS THAN MAXVALUE
);

CREATE TABLE merkle_roots (
  id BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  batch_date DATE NOT NULL UNIQUE,
  root_hash CHAR(64) NOT NULL,
  leaf_count INT UNSIGNED NOT NULL,
  published_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  external_ref VARCHAR(255) NULL -- reserved for external anchoring
) ENGINE=InnoDB;

```

14.5 Audit schema

```

❏
sql
CREATE TABLE audit_log (
  id BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  actor_user_id BIGINT UNSIGNED NULL,
  actor_role VARCHAR(40) NULL,
  action VARCHAR(60) NOT NULL, -- voucher.redeem, farm.verify, ...
  entity_type VARCHAR(40) NOT NULL,
  entity_id BIGINT UNSIGNED NULL,
  before_state JSON NULL,
  after_state JSON NULL,
  reason VARCHAR(255) NULL,
  ip_address VARBINARY(16) NULL,
  device_id VARCHAR(64) NULL,
  request_id CHAR(36) NULL,
  prev_hash CHAR(64) NULL,
  row_hash CHAR(64) NOT NULL, -- chained, tamper-evident
  created_at TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  INDEX idx_audit_entity (entity_type, entity_id, created_at),
  INDEX idx_audit_actor (actor_user_id, created_at),
  INDEX idx_audit_action (action, created_at)
) ENGINE=InnoDB;

```

The database user that the application runs as is granted `INSERT` and `SELECT` on `audit_log` and **no** `UPDATE` or `DELETE`. Deletion requires a separate administrative credential, used only under a documented retention procedure.

14.6 Data integrity rules enforced in the database

Rule	Mechanism
One national ID → one farmer	<code>UNIQUE KEY uq_national_id</code> on the salted hash
A voucher can never be over-redeemed	<code>CHECK (quantity_redeemed <= quantity)</code> + <code>SELECT ... FOR UPDATE</code>
A retried redemption cannot duplicate	<code>UNIQUE KEY uq_idempotency</code>
An allocation cannot be overdrawn	<code>CHECK (quantity_used <= quantity)</code>
A registrar cannot approve their own registration	<code>CHECK (approved_by <> registered_by)</code>
Flock event order is immutable	<code>UNIQUE KEY uq_flock_seq</code> + hash chain
A passport code is issued once	<code>UNIQUE KEY</code> on <code>passport_code</code>

15. API specification

15.1 Conventions

Aspect	Convention
Base URL	<code>https://api.poultrytrak.gov.gh/v1</code>
Format	JSON request/response, <code>UTF-8</code>
Auth	<code>Authorization: Bearer <JWT></code> (dealer terminals, mobile, integrations)
Idempotency	<code>Idempotency-Key: <uuid></code> required on all POSTs that mutate ledgers
Versioning	URI-versioned; breaking changes require a new major version
Errors	RFC 7807 problem+json
Rate limits	Per token and per IP; limits returned in <code>X-RateLimit-*</code> headers
Time	All timestamps ISO 8601 with offset; server stores UTC

Error envelope

```
json
{
  "type": "https://docs.poultrytrak.gov.gh/errors/voucher-already-redeemed",
  "title": "Voucher already redeemed",
  "status": 409,
  "detail": "This voucher was redeemed on 2026-08-19 at 10:42 by outlet PTD-AH-0042-01.",
  "code": "VOUCHER_ALREADY_REDEEMED",
  "request_id": "0f9c1a54-7b2e-4e21-9a1f-3d0c7e8b6a12",
  "meta": {
    "redeemed_at": "2026-08-19T10:42:11+00:00",
    "outlet_code": "PTD-AH-0042-01",
    "quantity": 5
  }
}
```

Error `detail` for a rejected redemption is deliberately informative: a dealer facing a farmer at the counter needs to be able to say *where and when* the voucher was already used, and the farmer needs that information to raise a dispute.

15.2 Endpoint summary

Authentication

Method	Path	Purpose
POST	<code>/auth/login</code>	Email/MSISDN + password, returns JWT + refresh token
POST	<code>/auth/otp/request</code>	Request OTP to a registered MSISDN
POST	<code>/auth/otp/verify</code>	Exchange OTP for a session
POST	<code>/auth/refresh</code>	Rotate access token
POST	<code>/auth/device/enrol</code>	Enrol a dealer terminal, returns device credentials
POST	<code>/auth/logout</code>	Revoke refresh token

Registry

Method	Path	Purpose
POST	/farmers	Create a farmer application
GET	/farmers/{poultrytrak_id}	Farmer detail (scope-restricted)
PATCH	/farmers/{id}/status	Suspend / reinstate / revoke (with reason)
POST	/farmers/{id}/farms	Add a farm
POST	/farms/{id}/verify	Submit field verification (GPS + photos)
POST	/farms/{id}/approve	District approval, mints PoultryTrak ID
GET	/farms/nearby?lat=&lng=&radius_m=	Spatial query, outbreak tracing

Vouchers

Method	Path	Purpose
POST	/vouchers/batch	Issue a batch against a district allocation
GET	/vouchers?farmer=	List a farmer's vouchers
POST	/vouchers/validate	Pre-flight check at the counter (no state change)
POST	/vouchers/redeem	Redeem (idempotent)
POST	/vouchers/redeem/offline-sync	Upload queued offline redemptions
POST	/vouchers/{id}/reverse	Reverse a redemption, with reason code
GET	/vouchers/{id}/history	Full audit trail for one voucher

Traceability

Method	Path	Purpose
POST	/flocks	Open a flock
POST	/flocks/{id}/events	Record a feed/vaccination/treatment/mortality event
GET	/flocks/{id}	Flock detail with event history
POST	/flocks/{id}/exit	Record an exit; runs withdrawal check
POST	/flocks/{id}/passports	Mint passports for an exit
GET	/passports/{code}	Passport record (public projection)
POST	/passports/bulk-verify	Verify a consignment (API key)

Reporting

Method	Path	Purpose
GET	/reports/reconciliation	Voucher reconciliation, filterable
GET	/reports/anomalies	Leakage and anomaly detection output
GET	/reports/coverage	Traceability coverage statistics
GET	/exports/{report}.csv	Streamed CSV export

15.3 Worked example — redemption

Request

☐

http

POST /v1/vouchers/redeem HTTP/1.1

Host: api.poultrytrak.gov.gh

Authorization: Bearer eyJhbGciOiJSUzI1NiIs...

Idempotency-Key: 8f14e45f-ea8f-4a1e-9f4a-2b1c3d4e5f60

Content-Type: application/json

```
{
  "voucher_code": "PTV-K7M2QX9A-4B7HN2PQ",
  "auth": { "method": "otp", "value": "483920" },
  "quantity": 5,
  "unit": "bag",
  "outlet_code": "PTD-AH-0042-01",
  "device_id": "TRM-AH-0042-01-A",
  "location": { "lat": 6.68848, "lng": -1.62443, "accuracy_m": 8.5 },
  "occurred_at": "2026-08-24T09:14:02+00:00"
}
```

Response — 201 Created

 json

```
{
  "redemption_id": 918273,
  "status": "redeemed",
  "voucher": {
    "code": "PTV-K7M2QX9A-4B7HN2PQ",
    "input_type": "FEED_STARTER",
    "entitled_quantity": 5,
    "redeemed_quantity": 5,
    "remaining_quantity": 0,
    "status": "redeemed"
  },
  "farmer": {
    "poultrytrak_id": "PT-GH-AH-KMA-004821-K",
    "display_name": "Ama Serwaa Boateng",
    "district": "Kumasi Metropolitan"
  },
  "value_ghs": 675.00,
  "geofence_ok": true,
  "receipt_url": "https://api.poultrytrak.gov.gh/v1/receipts/918273.pdf",
  "notifications": { "farmer_sms": "queued", "dealer_sms": "queued" }
}
```

Response — 409 Conflict (already redeemed) returns the problem+json envelope in §15.1, including where and when it was previously redeemed.

Response — 200 OK on retry. A repeat of the same `Idempotency-Key` returns the original `201` body with `"idempotent_replay": true` and creates nothing.

15.4 Worked example — public passport


http
`GET /v1/passports/9F2KQ70004821M`


json

```
{
  "passport_code": "9F2KQ70004821M",
  "verdict": "verified",
  "issued_at": "2026-08-14T11:02:00+00:00",
  "origin": {
    "farmer_display_name": "Ama Serwaa Boateng",
    "poultrytrak_id": "PT-GH-AH-KMA-004821-K",
    "district": "Kumasi Metropolitan",
    "region": "Ashanti",
    "farm_verified": true
  },
  "bird": {
    "type": "broiler",
    "breed": "Cobb 500",
    "hatch_date": "2026-06-20",
    "exit_date": "2026-08-14",
    "age_days": 55,
    "hatchery": "Akate Farms Hatchery"
  },
  "health": {
    "vaccinations": [
      { "disease": "Marek's", "date": "2026-06-20", "signed_by": "VET-AH-0091" },
      { "disease": "Newcastle", "date": "2026-06-27", "signed_by": "VET-AH-0091" },
      { "disease": "Gumboro", "date": "2026-07-04", "signed_by": "VET-AH-0091" },
      { "disease": "Newcastle (booster)", "date": "2026-07-18", "signed_by": "VET-AH-0091" }
    ],
    "treatments_recorded": 1,
    "withdrawal_compliant": true,
    "withdrawal_clear_date": "2026-08-02",
    "veterinary_signoff": { "officer_id": "VET-AH-0091", "date": "2026-08-13" }
  },
  "feed": [
    { "phase": "starter", "brand": "Agricare Starter", "subsidised": true, "period": "2026-06-20/2026-07-10" },
    { "phase": "grower", "brand": "Agricare Grower", "subsidised": true, "period": "2026-07-11/2026-07-31" },
    { "phase": "finisher", "brand": "Flour Mills Finisher", "subsidised": false, "period": "2026-08-01/2026-08-13" }
  ],
  "processing": { "processor": "Ashanti Poultry Processors Ltd", "date": "2026-08-14" },
  "integrity": {
    "flock_chain_head": "b1f0...9c2a",
    "merkle_root_date": "2026-08-15",
    "scan_count": 3,
  }
}
```

```
}
  "first_scan_at": "2026-08-16T08:30:11+00:00",
  "anomaly": false
}
```

16. Front-end specification (Carbon)

16.1 Carbon setup

```

html
<!-- IBM Plex, the Carbon type family -->
<link rel="stylesheet"
  href="https://fonts.googleapis.com/css2?family=IBM+Plex+Sans:wght@400;600&display=swap">
<!-- Carbon Design System, vanilla build -->
<link rel="stylesheet" href="/assets/carbon/carbon-components.min.css">
<script src="/assets/carbon/carbon-components.min.js" defer></script>
<!-- Project overrides -->
<link rel="stylesheet" href="/assets/css/poultrytrak.css">
```

Carbon assets are **self-hosted**, not CDN-loaded: the system must work on constrained connections and inside government networks with restricted egress, and a public-sector service should not have a third-party dependency in its critical rendering path.

16.2 Theme and tokens

Token	Value	Use
<code>\$interactive-01</code>	<code>#0f62fe</code>	Primary actions (Redeem, Approve, Verify)
<code>\$support-01</code> (error)	<code>#da1e28</code>	Rejected redemption, NOT VERIFIED verdict
<code>\$support-02</code> (success)	<code>#24a148</code>	Verified verdict, successful redemption
<code>\$support-03</code> (warning)	<code>#f1c21b</code>	Withdrawal notice, expiring vouchers
<code>\$support-04</code> (info)	<code>#0043ce</code>	Informational notices
<code>\$ui-01</code> / <code>\$ui-background</code>	<code>#f4f4f4</code> / <code>#ffffff</code>	Surfaces — Gray 10 theme
<code>\$text-01</code> / <code>\$text-02</code>	<code>#161616</code> / <code>#525252</code>	Primary/secondary text
Type	IBM Plex Sans	<code>productive-heading-*</code> , <code>body-short-*</code> , <code>body-long-*</code>
Grid	16-column Carbon grid	2x Grid, breakpoints sm 320 / md 672 / lg 1056 / xlg 1312
Spacing	<code>\$spacing-01</code> ... <code>\$spacing-13</code>	Never hard-code pixel spacing

A small accent palette derived from the national colours is permitted in the public passport view (verdict banner, badge) but **not** in the operational UI, where Carbon's semantic colour meanings must stay unambiguous.

16.3 Component mapping

Screen	Carbon components
Login	<code>Form</code> , <code>TextInput</code> , <code>PasswordInput</code> , <code>Button</code> , <code>InlineNotification</code>
Farmer registration wizard	<code>ProgressIndicator</code> , <code>Form</code> , <code>Select</code> , <code>DatePicker</code> , <code>FileUploader</code> , <code>Modal</code>
Farm verification	<code>Tile</code> , <code>FileUploader</code> (photos), <code>InlineNotification</code> (GPS accuracy), map embed
Farmer list / registry	<code>DataTable</code> with batch actions, <code>TableToolbarSearch</code> , <code>Pagination</code> , <code>Tag</code> (status)
Voucher issuance	<code>DataTable</code> , <code>NumberInput</code> , <code>Checkbox</code> (batch select), <code>Modal</code> (confirm)
Redemption terminal	<code>Search</code> (code entry), large <code>Button</code> , <code>Loading</code> , <code>ToastNotification</code> , <code>StructuredList</code> (result)
Flock record	<code>Tabs</code> , <code>Timeline</code> (custom, Carbon-styled), <code>Accordion</code> , <code>Tag</code>
Dashboards	<code>Tile</code> , <code>@carbon/charts</code> (line, bar, donut), <code>Dropdown</code> (filters)
Public passport	<code>Tile</code> , <code>InlineNotification</code> (verdict), <code>StructuredList</code> , <code>Accordion</code>
Admin settings	<code>Tabs</code> , <code>Toggle</code> , <code>NumberInput</code> , <code>Modal</code>

`@carbon/charts` is used for dashboards only; on the redemption path no chart library is loaded, keeping that page's payload minimal.

16.4 The redemption terminal — design rules

This screen is used hundreds of times a day, at a counter, sometimes outdoors, sometimes by a user who is not confident with technology. It gets its own rules:

One decision per screen. Scan → authenticate → confirm quantity → done. No screen asks two questions.

Verdict is unmissable. Success and failure states occupy the full width at the top, with an icon, a colour, and text — never colour alone (colour-blind users, glare, cheap screens).

Minimum 48 px touch targets, exceeding Carbon's defaults, for use with wet or gloved hands.

The result states *why*. "Already redeemed on 19 Aug at 10:42, PTD-AH-0042-01" — not "Error".

Total page weight under 150 KB on first load, cached thereafter; the flow must complete on 2G.

Works with the keyboard alone, because many outlets use a USB barcode scanner that emulates a keyboard.

Never blocks on the network for feedback. Local HMAC validation gives an instant preliminary verdict; the authoritative answer follows.

16.5 Accessibility

Target: **WCAG 2.1 AA**.

All Carbon components retain their built-in ARIA roles and keyboard handling; no re-implementation of native controls.

Every form field has a persistent visible label — not a placeholder acting as a label.

Errors are announced through `role="alert"` and linked to the offending field.

Contrast ratio $\geq 4.5:1$ for text, $\geq 3:1$ for UI boundaries; the Gray 10 theme is compliant by default and overrides must be checked.

Full keyboard operability with a visible focus ring; skip-to-content link on every page.

Language attribute set per page, ready for the multilingual phase.

16.6 Responsive and offline behaviour

Mobile-first: the officer and farmer flows are designed at 360 px width first, then enhanced.

The application is installable as a **PWA**; a service worker caches the shell, Carbon assets, and the offline redemption queue.

Every form that can be filled in the field is queued locally when offline and synchronised when connectivity returns, with a persistent Carbon notification showing the pending-sync count.

17. Security architecture

17.1 Threat model

#	Threat	Impact	Control
T1	Ghost farmer registration	Subsidy diversion	National ID dedupe, GPS verification, dual-control approval, random field audit
T2	Voucher code theft / sharing	Input diverted	Farmer OTP/PIN at redemption, farmer confirmation SMS, dispute channel
T3	Dealer redeems without releasing goods	State pays for nothing	Farmer confirmation SMS, dispute freeze, stock ledger reconciliation, physical spot checks
T4	Replay of a redemption request	Double payment	Idempotency key under unique index, row-level locking
T5	Forged voucher	Fraudulent claim	HMAC signature with per-season key; ledger is the settlement authority
T6	Counterfeit QR labels	Passport fraud	Non-enumerable codes, single-issue, duplicate-scan geographic anomaly detection
T7	Passport data scraping	Farmer privacy, competitive harm	Rate limiting, non-sequential codes, minimal public fields, no coordinates
T8	Insider record alteration	Loss of trust in the register	Append-only audit chain, no <code>UPDATE</code> / <code>DELETE</code> grant on audit, hash chains, published Merkle roots
T9	SQL injection / XSS	System compromise	Prepared statements everywhere, output escaping, CSP, no dynamic SQL string building
T10	Credential stuffing on officer accounts	Privileged access	Argon2id, MFA for all privileged roles, logout, IP-anomaly alerts
T11	Stolen dealer device	Unauthorised redemption	Device binding, PIN on device, remote revoke, offline limits
T12	Bulk data exfiltration by an authorised user	Mass privacy breach	Export quotas, watermarked exports, audit on every export, alerting on large reads

17.2 Authentication

Population	Primary factor	Second factor
Farmers	6-digit PIN (Argon2id)	OTP to registered MSISDN for redemption and sensitive changes
Dealers	Password + device binding	OTP per session; farmer OTP/PIN per transaction
Officers & admins	Password (Argon2id, min 12 chars, breach-list checked)	Mandatory TOTP
Integrations	mTLS or signed JWT	IP allow-list

Sessions: 30-minute idle timeout for privileged roles, 8-hour absolute cap, secure/HttpOnly/SameSite=Lax cookies, rotation on privilege change, full server-side invalidation on logout.

17.3 Authorisation

Role-based permissions with mandatory scope filtering. The repository base class requires an explicit scope on every query; a query without a scope throws rather than returning national data by default. Permission checks happen in the service layer, not the view — a hidden button is not an access control.

```

.php
// Illustrative: scope is not optional.
final class FarmerRepository extends ScopedRepository
{
    public function findByPoultryTrakId(string $id, Scope $scope): ?Farmer
    {
        $sql = "SELECT * FROM farmers
            WHERE poultrytrak_id = :id
            {$this->scopeClause($scope, 'region_id', 'district_id')}
            LIMIT 1";
        return $this->fetchOne($sql, ['id' => $id] + $scope->bindings());
    }
}
```

17.4 Cryptography

Purpose	Algorithm
Password / PIN hashing	Argon2id (<code>memory_cost</code> 65536, <code>time_cost</code> 4, <code>threads</code> 2)
Voucher signature	HMAC-SHA256, truncated to 40 bits for the printed tag, full tag verified server-side
Data at rest (national ID, MFA secrets)	AES-256-GCM, keys in a KMS or an HSM-backed store, never in the codebase or <code>.env</code>
Event/audit chaining	SHA-256
API tokens	RS256 JWT, short-lived access + rotating refresh
Transport	TLS 1.3, HSTS with preload, no TLS 1.0/1.1
Randomness	<code>random_bytes()</code> only — never <code>rand()</code> , <code>mt_rand()</code> , or <code>uniqid()</code> for anything security-relevant

Key rotation: season signing keys rotate per programme; TLS certificates automated; database encryption keys rotate annually with re-encryption jobs.

17.5 Application hardening

All database access via PDO prepared statements with typed bindings. Dynamic SQL construction is prohibited by code review and detected in static analysis.

Output escaped at render time via a single `e()` helper; raw echo of user data fails review.

CSRF tokens on every state-changing form, verified with `hash_equals`.

Content Security Policy: `default-src 'self'`, no inline scripts, nonce-based where unavoidable.

Uploads: MIME sniffing, extension allow-list, EXIF stripping, re-encoding of images, storage outside the web root, randomised filenames, size caps.

Security headers: `X-Content-Type-Options`, `Referrer-Policy`, `Permissions-Policy`, `X-Frame-Options: DENY`.

Dependency scanning in CI; no known-vulnerable packages permitted in a release.

`display_errors=Off` in production, structured logging with PII redaction.

17.6 Audit and forensics

Every privileged action writes an `audit_log` row with before/after state, actor, reason, request ID and device.

Rows are hash-chained: `row_hash = SHA-256(prev_hash || canonical(row))`. A nightly verifier walks the chain and alerts on the first mismatch. Log retention: 7 years, matching public financial record requirements.

17.7 Reserved: external anchoring

Daily Merkle roots (§10.5) are stored in `merkle_roots` with an `external_ref` column deliberately left empty in v1. If a later phase requires third-party-verifiable immutability — typically an export market requirement — those roots can be published to an external notarisation service or public ledger without any change to the passport or event schema. This is a stated design option, not an implemented feature; the documentation should not claim otherwise.

18. Offline, USSD and low-connectivity strategy

18.1 Why this is a first-class concern

A traceability system that only works where the network works will produce a register biased toward peri-urban farms, which is exactly the population least in need of the intervention. Coverage in poultry-producing areas of the Ashanti, Bono, Brong Ahafo and Northern zones is uneven, and a redemption that cannot complete at the counter sends the farmer home empty-handed.

18.2 Offline redemption

The dealer terminal (PWA) can complete a redemption without connectivity, under strict limits:

Control	Value (configurable per programme)
Local signature verification	Mandatory — offline redemption of an unsigned code is impossible
Offline queue cap	25 redemptions or GHS 15,000, whichever comes first
Maximum offline window	48 hours; after that the terminal refuses offline mode until it syncs
Farmer authentication offline limited verifier	PIN only (OTP requires network); PIN verified against a device-held, salted, time-
Value cap per offline transaction	Configurable; high-value inputs are excluded from offline mode entirely
Conflict resolution	Server is authoritative. If a voucher was redeemed elsewhere first, the offline record is rejected, both parties are notified, the dealer's claim excludes it, and the incident feeds the dealer risk score
Sync	Automatic on reconnection; the terminal shows an unmissable pending-sync banner until the queue is empty

The rejection path must be designed as carefully as the success path, because it is where disputes originate.

18.3 USSD

For farmers with feature phones, a USSD short code provides:



*XXX#

1. My PoultryTrak ID
2. My vouchers
 > list with code, input, quantity, expiry
3. Confirm a redemption (enter code shown by dealer)
4. Report a problem (dispute categories 1-5)
5. Check a passport code
6. Register interest (routes to district officer)
7. Change my PIN

USSD sessions are stateless on the client, so session state is held server-side in Redis with a short TTL, keyed by MSISDN and session ID.

18.4 SMS

Trigger	Message
Registration approved	PoultryTrak ID + PIN setup instructions
Voucher issued	Code, input, quantity, expiry date
Redemption completed	Dealer name, input, quantity, and how to dispute
Voucher expiring	7-day and 1-day warnings
Dispute status change	Outcome and next step
Quarantine order	Instructions and the district vet contact

All SMS templates are short, in plain language, and avoid jargon. Message content is treated as public — no sensitive data beyond what the farmer already knows.

19. Integrations

System	Direction	Purpose	Mechanism
National identity register	Outbound query	Verify national ID against name and date of birth	REST + mTLS; graceful degradation to officer attestation if unavailable
SMS / USSD aggregator	Both	OTP, notifications, USSD menu	HTTP callbacks + outbound API, delivery receipts stored
Mobile money	Outbound	Farmer co-payment on partially subsidised inputs; dealer reimbursement	Provider API; PoultryTrak records references only and never stores payment credentials
GIS basemap	Inbound	District boundaries, exclusion polygons, map tiles	Tile server or offline vector tiles for field devices
Veterinary Services Directorate	Both	Notifiable disease reporting, vaccination campaign data	Scheduled export + webhook on notifiable events
Feed mills & hatcheries	Inbound	Batch numbers, delivery confirmations	API or CSV upload against a defined schema
Processors / abattoirs	Both	Consignment receipt, processing dates, pack code minting	API
GS1 / barcode standards body	Reference	Alignment of identifiers where export requires GS1-compliant coding	Standards mapping, not a runtime dependency
Certification / export authority	Outbound	Consignment dossiers for export documentation	Signed PDF + verification API
Financial institutions (<i>phase 3</i>)	Outbound, consented	Farmer-authorised production history for credit assessment	Scoped, time-limited, farmer-granted token

Integration principle: **PoultryTrak degrades, it does not fail.** Every external dependency has a defined fallback, and no external outage may block a farmer from redeeming an entitlement they legitimately hold.

20. Non-functional requirements

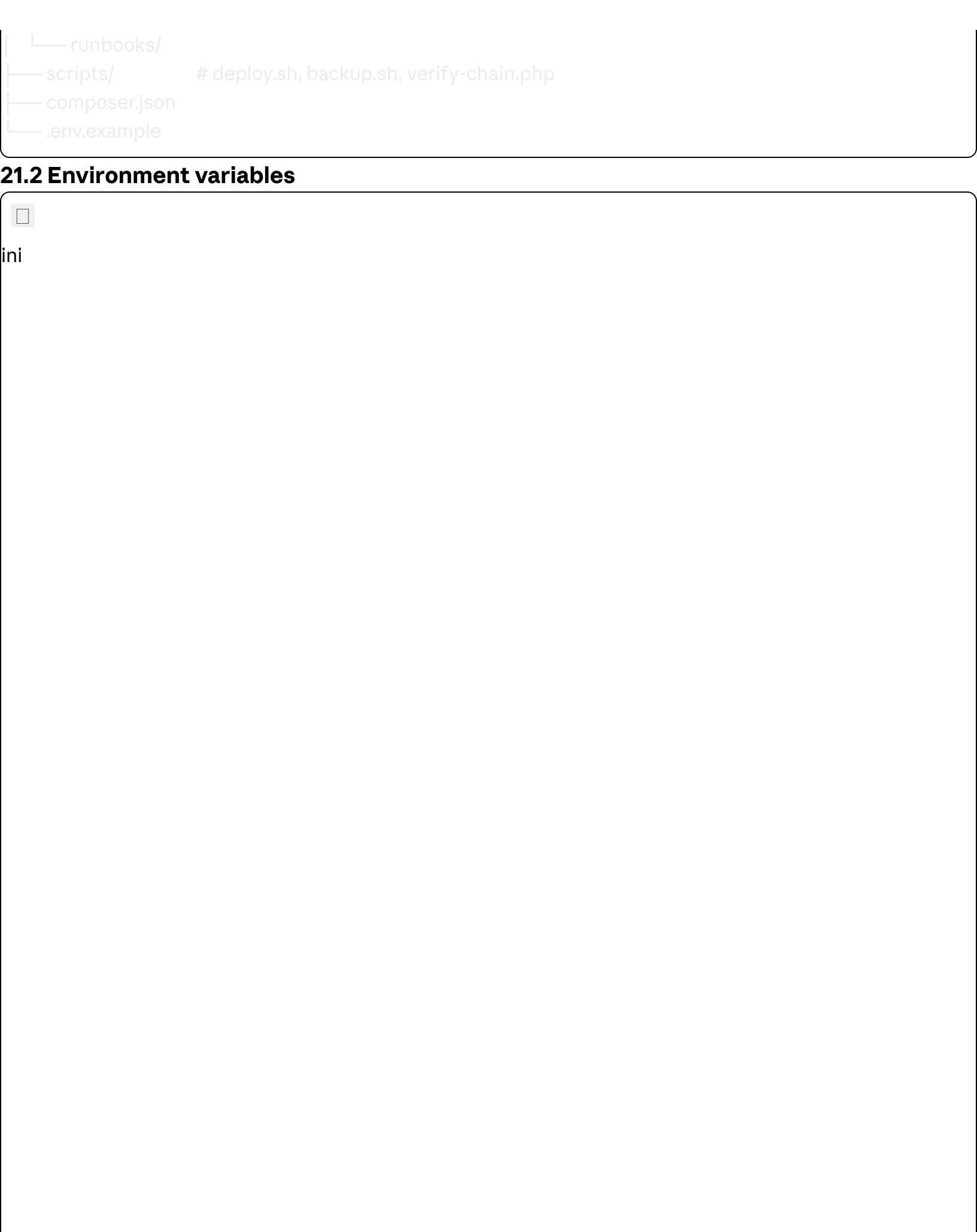
Category	Requirement
Availability	99.5% monthly for core services; 99.9% for the public passport endpoint
Performance	Voucher validation p95 < 400 ms; redemption p95 < 800 ms; passport lookup p95 < 300 ms; dashboards < 2 s
Page weight	Redemption terminal < 150 KB first load; public passport < 200 KB including QR
Scale (year 3)	60,000 farmers · 90,000 farms · 400,000 flocks/year · 30M passports/year · 5M redemptions/year
Concurrency	500 concurrent redemption sessions at peak input-distribution windows
Data retention	Registry: indefinite while active + 10 years. Vouchers and audit: 7 years. Passport public records: 5 years post-exit. Scan telemetry: 24 months
Backup	Hourly incremental, nightly full, 30-day point-in-time recovery; monthly restore drill with a written result
RPO / RTO	RPO 15 minutes; RTO 4 hours
Browser support	Chrome/Android 90+, Safari 14+, Firefox 90+; graceful degradation to no-JS for core read paths
Device floor	Android 8, 2 GB RAM, 5-inch screen, 2G fallback
Localisation	English at launch; message catalogue externalised for Twi, Ewe, Ga, Dagbani, Hausa
Observability	Structured JSON logs, request IDs propagated end to end, uptime and latency dashboards, alerting on redemption error rate, sync backlog and chain-verification failure

21. Project structure & environment

21.1 Directory layout



- └─ runbooks/
- └─ scripts/ # deploy.sh, backup.sh, verify-chain.php
- └─ composer.json
- └─ .env.example



A screenshot of a terminal window. The top part shows a directory listing with the following items: 'runbooks/' (indicated by a vertical line), 'scripts/' (indicated by a vertical line, with a comment '# deploy.sh, backup.sh, verify-chain.php' to its right), 'composer.json' (indicated by a vertical line), and '.env.example' (indicated by a vertical line). Below this listing, the text '21.2 Environment variables' is displayed in a bold font. The terminal window has a title bar at the top with a small square icon on the left and the text 'ini' on the right.

```
APP_ENV=production
APP_URL=https://poultrytrak.gov.gh
APP_KEY=          # base64, 32 bytes, from a secret manager
APP_DEBUG=false

DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=poultrytrak
DB_USERNAME=pt_app      # least privilege: no DROP, no DELETE on audit_log
DB_PASSWORD=
DB_SSL_CA=/etc/mysql/ca.pem

REDIS_HOST=127.0.0.1
REDIS_PORT=6379

KMS_PROVIDER=aws|vault|local-hsm
KMS_KEY_ID=          # voucher signing + field encryption

SMS_PROVIDER=
SMS_API_KEY=
SMS_SENDER_ID=POULTRYTRAK
USSD_SHORTCODE=

NATIONAL_ID_API_URL=
NATIONAL_ID_CLIENT_CERT=

QR_BASE_URL=https://verify.poultrytrak.gov.gh/p/
PASSPORT_CODE_KEY=    # keyed permutation for non-enumerable codes

VOUCHER_OFFLINE_MAX_COUNT=25
VOUCHER_OFFLINE_MAX_VALUE_GHS=15000
VOUCHER_OFFLINE_MAX_HOURS=48
GPS_MAX_ACCURACY_M=25
FARM_DUPLICATE_RADIUS_M=50
```

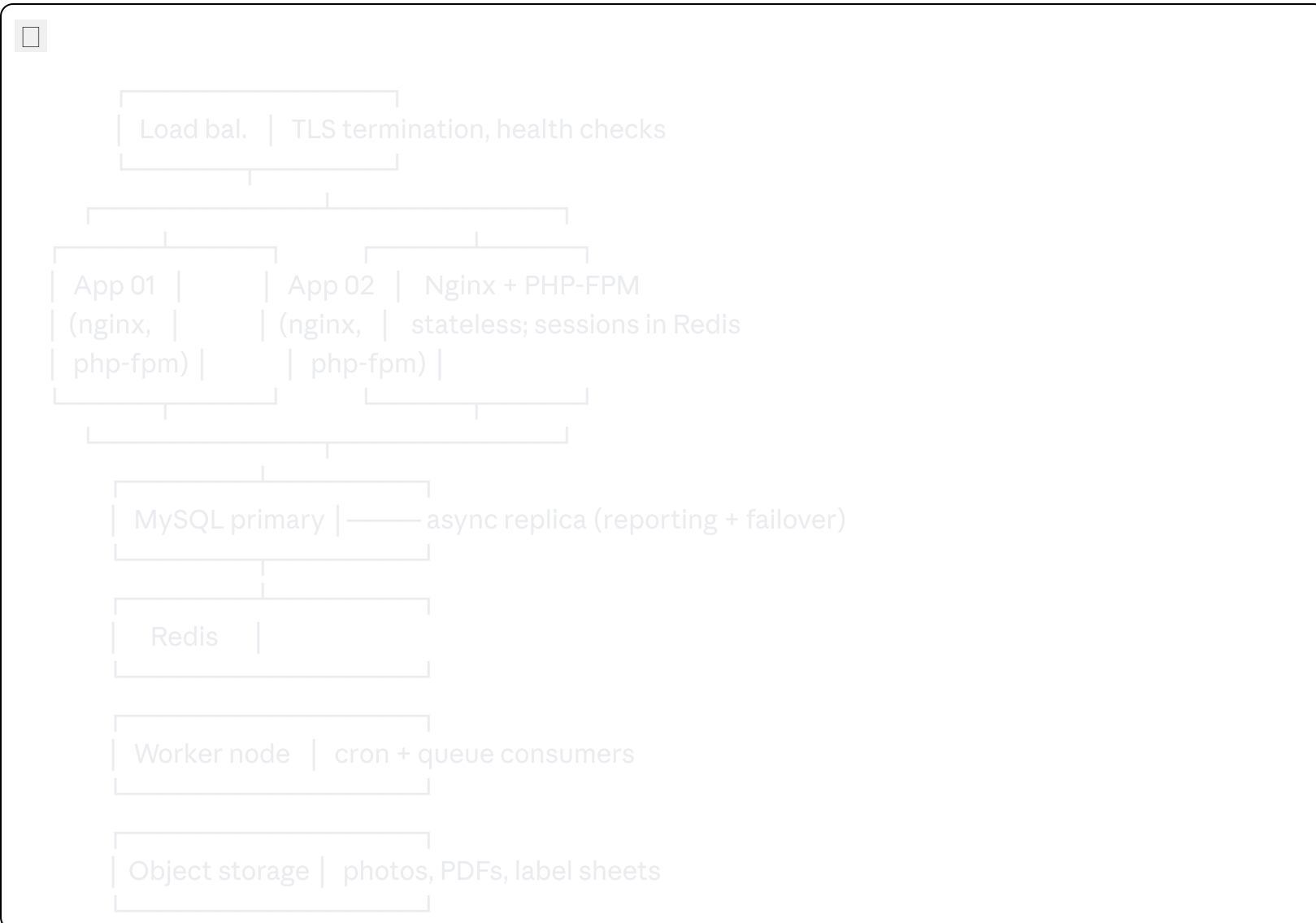
`.env` is never committed. In production, secrets are injected from a secret manager rather than a file on disk wherever the hosting environment allows it.

22. Deployment & operations

22.1 Environments

Environment	Purpose	Data
Local	Developer machines	Synthetic seed data only
CI	Automated test runs	Ephemeral
Staging	UAT, officer training, integration testing	Anonymised or synthetic — never live personal data
Production	Live service	Live

22.2 Reference production topology



Minimum viable single-node configuration for pilot: 4 vCPU, 8 GB RAM, 200 GB SSD, with nightly off-site backups. The architecture scales horizontally on the app tier without redesign because application servers hold no state.

22.3 Scheduled jobs

Job	Schedule	Purpose
<code>ExpireVouchers</code>	Hourly	Move past-expiry vouchers to <code>expired</code> , return quantity to the district pool
<code>SendExpiryReminders</code>	Daily 07:00	7-day and 1-day SMS warnings
<code>VerifyChains</code>	Nightly 01:00	Recompute flock and audit hash chains, alert on mismatch
<code>BuildMerkleRoot</code>	Nightly 02:00	Batch closed-flock heads into a daily root
<code>ReconcileRedemptions</code>	Nightly 03:00	Match redemptions to stock movements, build claim lines, flag anomalies
<code>DetectAnomalies</code>	Every 15 min	Velocity, geofence, duplicate-scan detection
<code>RefreshDashboards</code>	Every 10 min	Materialise dashboard aggregates
<code>ReverificationSweep</code>	Weekly	Flag farms past re-verification date
<code>PurgeExpiredSessions</code>	Hourly	Session and OTP hygiene
<code>BackupVerify</code>	Daily	Restore a sample backup and assert row counts

22.4 Release process

Feature branch → pull request → mandatory review by a second engineer.

CI: PSR-12 lint, static analysis, unit + integration tests, dependency vulnerability scan, migration dry-run.

Merge to `main` → automatic deploy to staging → UAT sign-off.

Production deploy in a maintenance window for schema changes; zero-downtime otherwise.

Migrations are **forward-only and additive**. Destructive changes are two-phase: deploy code that tolerates both shapes, backfill, then drop in a later release.

Every release is tagged, with a changelog entry and a documented rollback step.

22.5 Monitoring and alerting

Signal	Alert threshold
Redemption error rate	> 2% over 15 min
Redemption p95 latency	> 1.5 s over 10 min
Offline sync backlog	Any terminal > 24 h unsynced
Chain verification	Any mismatch — critical, page immediately
SMS delivery failure rate	> 10% over 30 min
Failed login spike	5× baseline
Database replication lag	> 30 s
Disk usage	> 80%
Backup job	Any failure — critical

22.6 Runbooks

Written runbooks are required, at minimum, for: suspected fraudulent dealer, duplicate passport code detected in the field, national ID service outage, SMS gateway outage, database failover, restoring from backup, key rotation, and mass voucher reversal after an issuance error.

23. Testing strategy

23.1 Test layers

Layer	Coverage target	Focus
Unit	≥ 80% of domain code	Entitlement calculation, voucher state machine, withdrawal calculator, ID check character, hash chain, code generation
Integration	All repositories and services	Transactions, locking, constraint enforcement against a real MySQL instance
Feature / end-to-end	All critical journeys	Registration → approval → voucher issuance → redemption → flock → passport → public scan
Security	Each release	SAST, dependency scan, plus an independent penetration test before go-live and annually
Performance	Before each major release	Load test the redemption path at 3× expected peak
Accessibility	Each release	Automated axe scan plus manual keyboard and screen-reader pass on core screens
Field / UAT	Each phase	Real officers, real dealers, real farms, real network conditions

23.2 Critical test cases (must never regress)

ID	Scenario	Expected
TC-01	Redeem the same voucher twice, sequentially	Second attempt rejected with prior redemption details
TC-02	Two concurrent redemption requests for one voucher	Exactly one succeeds; the other gets a clean conflict
TC-03	Retry with the same idempotency key	Original result replayed; no second redemption row
TC-04	Redeem at a dealer outside the permitted region	Rejected
TC-05	Redeem after expiry	Rejected; entitlement returned to pool
TC-06	Redeem for a suspended farmer	Rejected
TC-07	Redeem more than the entitled quantity	Rejected at validation
TC-08	Partial redemption	Residual voucher issued for the exact balance; farmer notified
TC-09	Offline redemption of a voucher already redeemed online	Rejected on sync; both parties notified; excluded from claim
TC-10	Offline queue exceeds cap	Terminal refuses further offline redemptions
TC-11	Register a farm within the duplicate radius of a verified farm	Blocked, flagged for officer review
TC-12	GPS accuracy above threshold	Verification rejected
TC-13	Same national ID registered twice	Blocked by unique constraint before any ID is minted
TC-14	Registering officer attempts to approve their own registration	Blocked

ID	Scenario	Expected
TC-15	Exit a flock inside a withdrawal period	Passport minting blocked unless a vet records an override; passport then shows an amber notice
TC-16	Mint two passports with the same code	Impossible — unique constraint
TC-17	Alter a flock event row directly in the database	Nightly chain verification raises a critical alert
TC-18	Scan the same passport code in two distant regions within an hour	Anomaly flagged, verdict shows a warning
TC-19	Sequential enumeration of passport codes	Rate-limited and alerted
TC-20	District issues vouchers exceeding its allocation	Blocked at issuance, not at reconciliation
TC-21	An officer queries a farmer outside their district	Empty result; attempt audited
TC-22	Complete a redemption on a 2G connection	Succeeds within the latency budget

24. Governance, legal & data protection

24.1 Institutional governance

Body	Responsibility
Programme steering committee	Policy, allocation envelopes, escalation of systemic fraud findings
MoFA / Directorate of Animal Production	Registry ownership, extension officer deployment
Veterinary Services Directorate	Health records, sign-off authority, disease response
Programme secretariat	Day-to-day operations, dealer accreditation, user management
Independent audit	Read-only access, quarterly reconciliation review
Farmer association representation	Grievance oversight, feedback on field usability

24.2 Data protection

The system processes personal data of farmers and officers and must comply with Ghana's **Data Protection Act, 2012 (Act 843)** and the requirements of the Data Protection Commission.

Principle	Implementation
Lawful basis and consent	Registration includes an explicit, plain-language consent notice covering what is collected, why, who sees it, and what appears publicly. Consent is recorded with a timestamp
Purpose limitation	Registry data is used for programme administration, disease control and traceability only. Any secondary use requires fresh consent
Data minimisation	Only fields with a defined operational use are collected; the public passport exposes the minimum set in §10.4
Accuracy	Farmers can view their record and request correction through an officer or the app; corrections are audited
Storage limitation	Retention schedule in §20, with automated purge jobs
Security	Encryption in transit and at rest, RBAC, audit logging, breach detection
Data subject rights	Access, correction, and objection handled through a documented process with a service-level target of 14 days
Breach notification	Documented procedure with notification to the Data Protection Commission and affected individuals within the statutory window
Registration	The controller registers with the Data Protection Commission before processing begins

24.3 Specific privacy decisions

Farm coordinates are never public. They are visible only to scoped officials and are excluded from every export that leaves the system.

National ID numbers are encrypted at rest and never displayed in full; the interface shows a masked form.

The public passport shows a display name, which the farmer chooses and may set to a business name rather than a personal name.

Scan telemetry uses hashed IPs and coarse geography, sufficient for anomaly detection but not for tracking individuals.

Exports are watermarked and audited, so a leaked dataset can be traced to the export that produced it.

24.4 Fraud response policy

Findings are graded and handled consistently: administrative error → correction with audit note; negligence → warning and mandatory retraining; deliberate diversion → suspension, claim forfeiture, de-accreditation and referral to law enforcement. Every grade requires evidence from the audit trail, a named decision-maker,

and a documented appeal route. A system that can detect fraud but has no consistent consequence for it will not deter it.

24.5 Intellectual property and licensing

Carbon Design System is licensed under Apache 2.0, and all Composer dependencies listed in §7.2 carry permissive licences (MIT/BSD/Apache); attribution files are bundled with the deployment. Source code ownership, escrow, and the right of the ministry to engage another maintainer must be settled in the development contract before the build begins — this is the single most common cause of stranded public-sector systems.

25. Implementation roadmap

Phase 0 — Foundation (6 weeks)

Requirements validation with MoFA and farmer associations · ADRs · database schema and migrations · Carbon design system set-up and component library · authentication, RBAC and audit logging · CI pipeline and environments. **Exit criteria:** a user can log in, roles are enforced, every action is audited.

Phase 1 — Registry (8 weeks)

Farmer application, identity check, GPS field verification, dual-control approval, PoultryTrak ID minting, ID card generation, farm records, duplicate detection, district registry dashboards, officer mobile flows. **Exit criteria:** 500 farmers registered and verified in two pilot districts, with a measured duplicate-detection rate.

Phase 2 — Vouchers (10 weeks)

Programme and allocation configuration, entitlement engine, voucher issuance and signing, dealer accreditation and outlets, device enrolment, redemption terminal, OTP/PIN authentication, offline queue and sync, disputes, reconciliation and claims. **Exit criteria:** one full input-distribution cycle completed in the pilot districts with reconciliation variance below 3%.

Phase 3 — Traceability (10 weeks)

Flock registration, event capture, vaccination and treatment records with withdrawal enforcement, hash chaining, exit workflow, passport minting, QR label generation and printing, public verification portal, bulk consignment verification. **Exit criteria:** 50,000 birds passported and verified end-to-end by a participating supermarket or processor.

Phase 4 — Intelligence & scale (8 weeks)

Anomaly detection, dealer risk scoring, disease surveillance and spatial contact tracing, national dashboards, exporter dossiers, Merkle root publication, performance hardening, penetration test remediation. **Exit criteria:** load-tested at 3× projected peak; independent security assessment closed out.

Phase 5 — National rollout (*continuous*)

Regional onboarding waves, officer and dealer training, multilingual UI, support desk, and a monitoring-and-evaluation baseline against §3.2.

Recommended pilot geography: two contiguous districts in a high-density poultry zone with mixed connectivity, at least three accredited dealers, one hatchery and one processor — enough to exercise every integration point without national exposure.

26. Risks and mitigations

#	Risk	Likelihood Impact		Mitigation
R1	Low digital literacy among farmers slows adoption	High	High	USSD and SMS channels; officer-assisted flows; nothing critical requires a smartphone; training built into registration
R2	Dealers resist because leakage was profitable	Medium	High	Prompt reimbursement is the incentive; accreditation is the lever; publish payment turnaround times and meet them
R3	Network outages block redemption	High	Medium	Offline-capable terminal with defined limits (§18.2)
R4	National ID integration unavailable or slow	Medium	Medium	Officer attestation fallback with document capture; retro-validation when the service returns
R5	Collusion between farmer and dealer to redeem and resell	Medium	High	Physical spot checks, stock reconciliation, risk scoring, capacity-linked entitlement, flock records that must show the input was used
R6	Counterfeit QR labels in the market	Medium	High	Non-enumerable single-issue codes, duplicate-scan geographic detection, buyer education, tamper-evident label stock
R7	Political pressure to bypass verification for specific beneficiaries	Medium	Critical	Hard system constraints rather than policy-only controls; every override is logged, attributed and reportable to audit
R8	Insufficient officer devices or airtime	High	High	Budget line for devices and data as part of the programme, not an afterthought; low-bandwidth design reduces cost
R9	Data breach of the farmer register	Low	Critical	Encryption, least privilege, export controls, monitoring, penetration testing, breach runbook
R10	Vendor lock-in / stranded system	Medium	High	Source code ownership and escrow agreed up front (§24.5); mainstream stack; documented architecture; local skills
R11	Scope creep into a full farm-management platform	High	Medium	Explicit non-goals (§3.3); change control through the steering committee

#	Risk	Likelihood Impact		Mitigation
R12	Passport promises exceed what records can support	Medium	High	Verdict states include amber and red; withdrawal enforcement cannot be bypassed silently; documentation states flock-level granularity honestly (§6.4)
R13	Buyers do not adopt scanning, so traceability earns no premium	Medium	High	Engage two or three formal buyers during Phase 3, not after; bulk verification and exporter dossiers built for their existing workflows
R14	Sustained funding ends after the pilot	Medium	Critical	Design low running costs; quantify recovered leakage as the funding argument; plan a modest verification-service fee for commercial buyers

27. Glossary

Term	Definition
PoultryTrak ID	Unique, permanent, non-transferable identifier issued to a verified farmer
Farm code	Identifier for one physical farm belonging to a farmer, derived from their PoultryTrak ID
Flock	A batch of birds placed together, sharing housing, feed and vaccination programme — the unit of traceability record
Passport	A public, verifiable record for one bird, crate or pack, resolving to its flock's full history
Passport code	The non-enumerable public code encoded in the QR and printed in human-readable form
Voucher	A signed, single-use entitlement to a specific quantity of a specific subsidised input, bound to one farmer
Residual voucher	The balance voucher automatically issued after a partial redemption
Reversal	A compensating entry that cancels a redemption without deleting it
Accredited dealer	An agro-input business authorised to redeem vouchers for specified input classes
Withdrawal period	The interval after a veterinary treatment during which a bird may not enter the food chain
Withdrawal-clear date	The computed date on which a treated flock becomes eligible for exit
Geofence check	Comparison of a redemption's GPS position against the dealer outlet's registered location
Idempotency key	A client-supplied unique key ensuring a retried request cannot create a duplicate transaction
Hash chain	A sequence of records where each stores a hash of its predecessor, making silent alteration detectable
Merkle root	A single hash summarising a batch of records, publishable as tamper-evidence
Verdict	The passport's headline status: <code>verified</code> , <code>notice</code> , or <code>not verified</code>
Scope	The geographic and role boundary limiting what data a user may query

Term	Definition
Carbon DS	IBM's Carbon Design System — the front-end component and styling framework
PWA	Progressive Web App — a web application installable to the home screen with offline capability
MSISDN	A mobile phone number in international format
Notifiable disease	A disease that must be reported to veterinary authorities by law

Appendix A — Change control

Version	Date	Author	Change
1.0	2026-08-24	—	Initial specification

Appendix B — Open questions for the sponsor

Is a national identity verification API available to the programme, and under what data-sharing agreement?

What is the committed reimbursement turnaround for dealer claims? This number determines dealer willingness to participate more than any technical feature.

Who holds legal ownership of the register — MoFA, the Veterinary Services Directorate, or a joint body — and who is the registered data controller?

What is the accreditation and de-accreditation authority for dealers, and what is the appeal route?

Which two districts are the pilot, and are the officers there already equipped with devices?

Which formal buyers will commit to scanning during Phase 3? Traceability without a buyer who checks it is a cost with no return.

What is the budgeted funding horizon, and what is the intended sustainability model beyond it?

Is there an existing farmer register that must be migrated, and what is its quality?